

High load ML

Vladislav Goncharenko

Video recommendations
team lead, Dzen



MIPT,
spring 2025

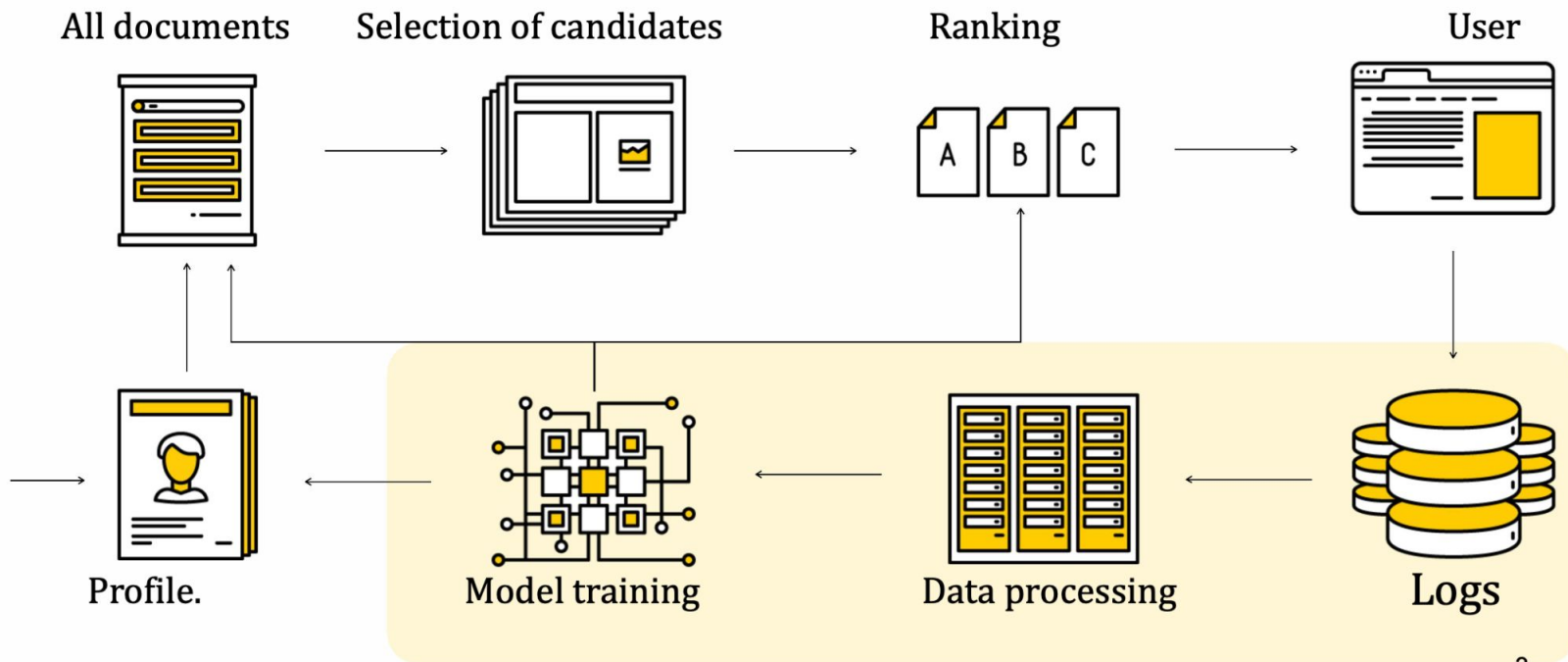
Typical service



- 10^7 documents
- 10^6 users per day
- 1-10k requests per second (RPS) for recommendations

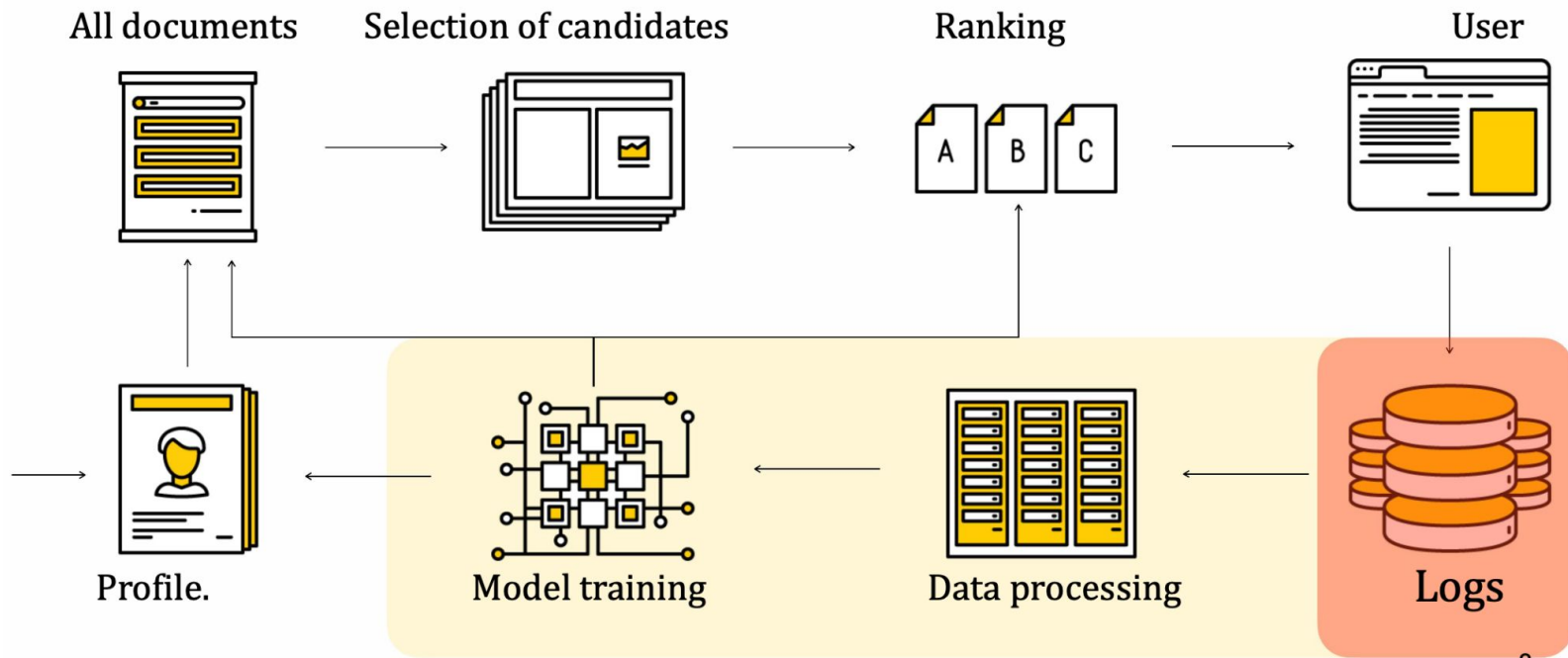
What could possibly go wrong? 🤔

The real system





Logging





Logging

- To train algorithms, information about user actions is stored in a structured form
- At a minimum, you will need a bundle of "user-item-rating-timestamp"
- Logs should be stored forever because this is unique information about your users

<code>_version</code>	<code>String optional</code>
<code>client_info</code>	Struct <i>optional</i> <code>client_type : String</code> <code>client_version : String</code>
<code>event_id</code>	<code>String optional</code>
<code>event_type</code>	<code>String optional</code>

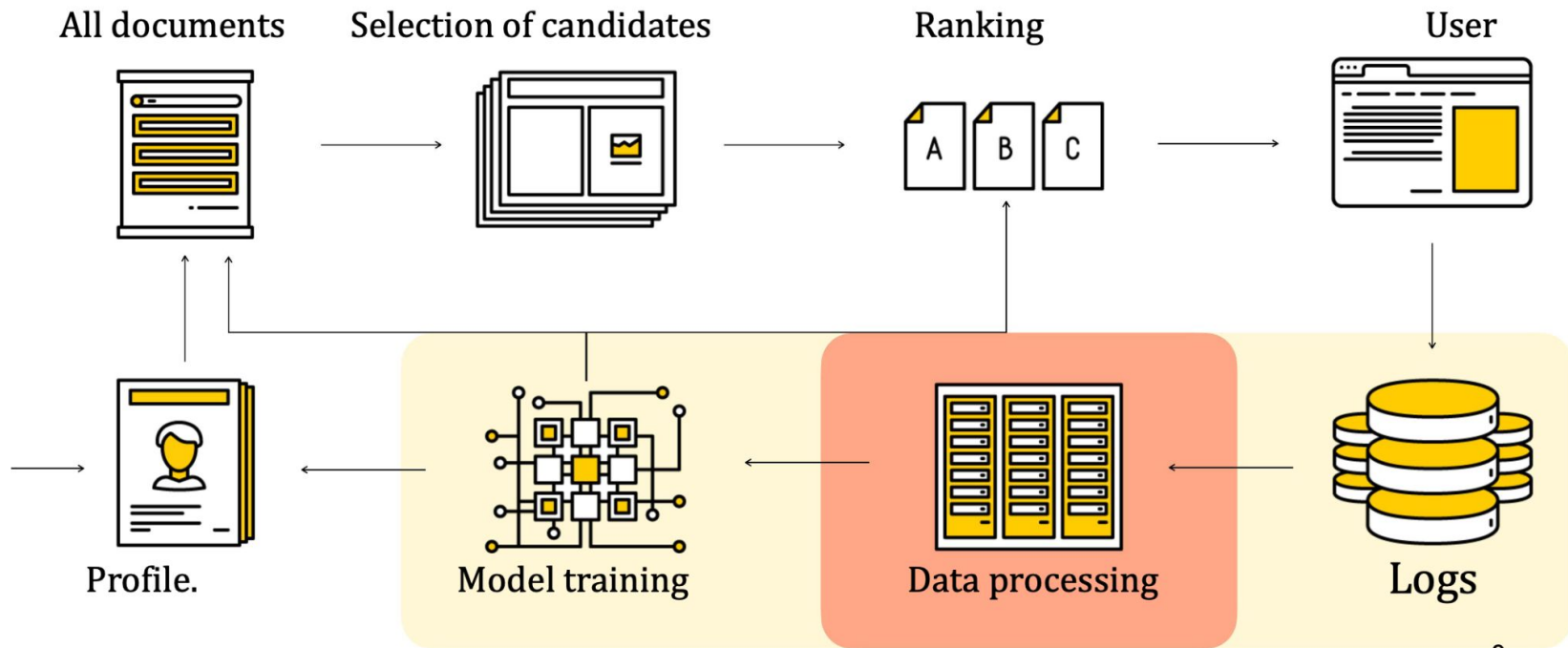
Data processing

girafe
ai

01



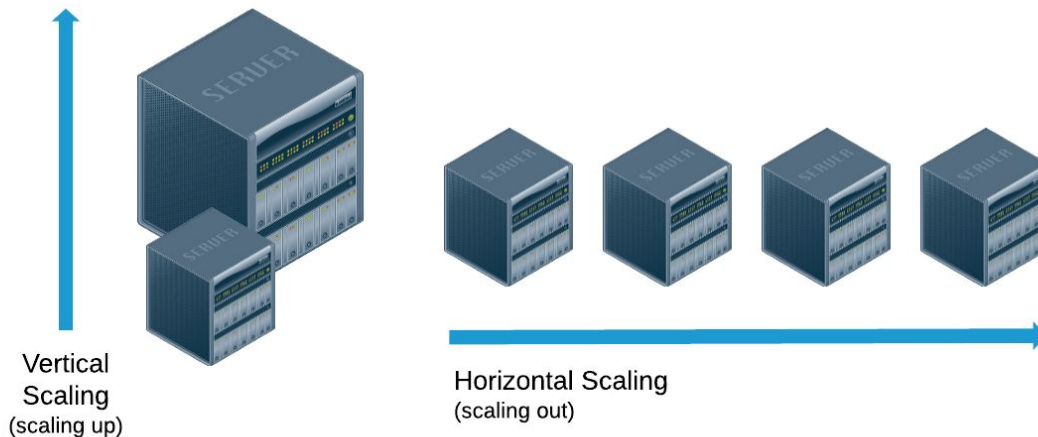
Data processing



Scalability

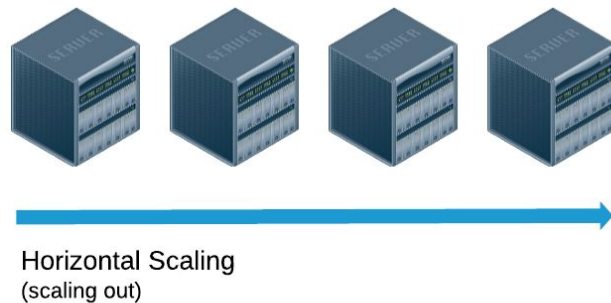
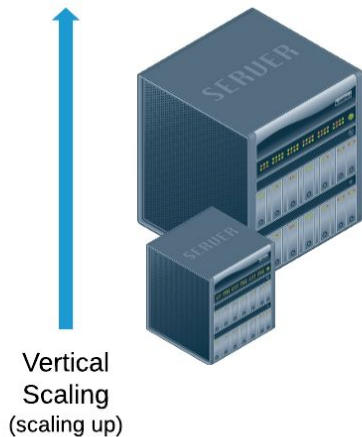


- When there was less data and they were more structured, they could be put into a relational database, having thought out the data schema and a list of possible queries to them in advance
- To speed up processing, we need a more powerful server - **vertical scalability**
- In the MapReduce paradigm, more average servers are needed to speed up processing (**horizontal scalability**), which is much cheaper



Scalability

- vertical scaling is limited
- horizontal is less costly
- horizontal is more robust to breaks



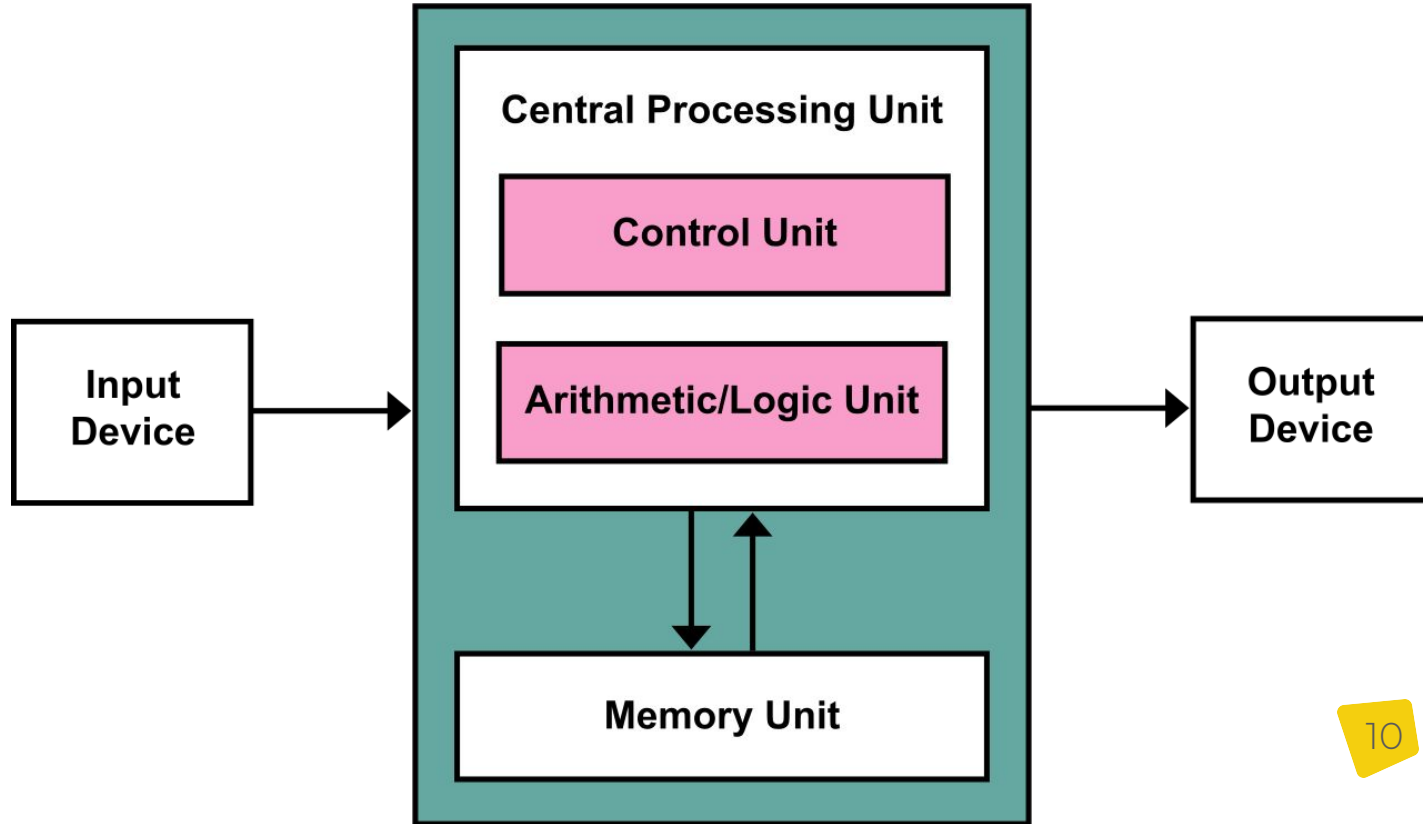
von Neumann architecture



Computer architecture proposed in 1945.

Up to today computer systems follow this design.

https://en.wikipedia.org/wiki/Von_Neumann_architecture





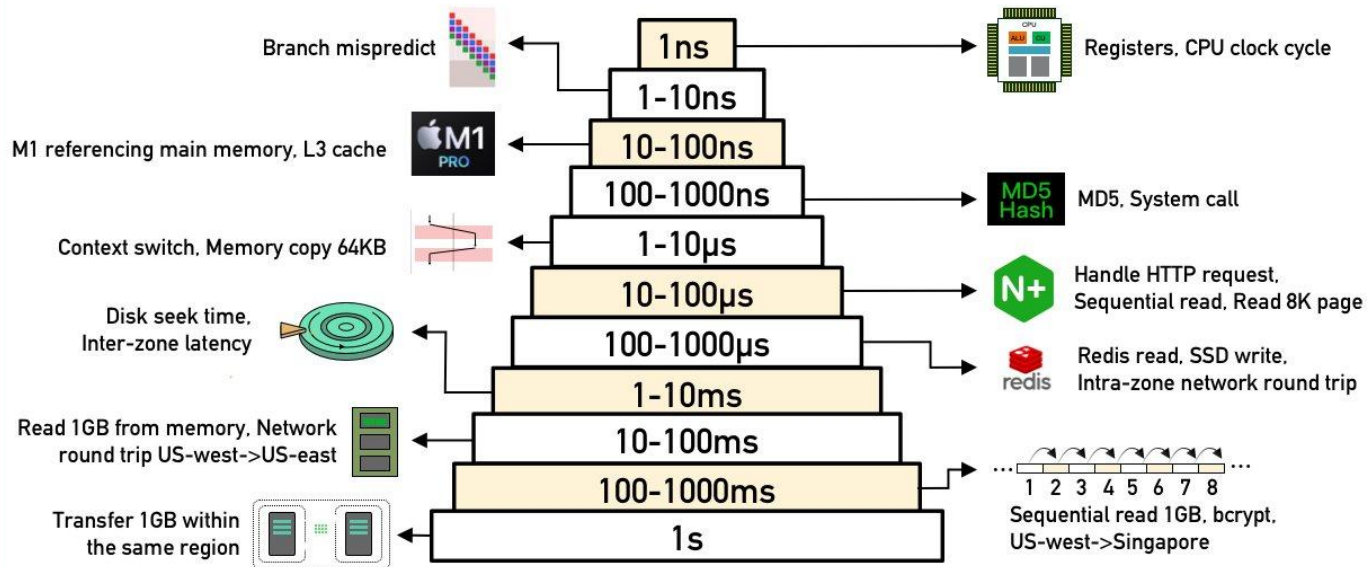
Data Locality

[Ulrich Drepper, What every programmer should know about memory \(2007\)](#)

[Stackoverflow discussion](#)

Latency numbers for the 2020s

ByteByteGo.com





Data Locality

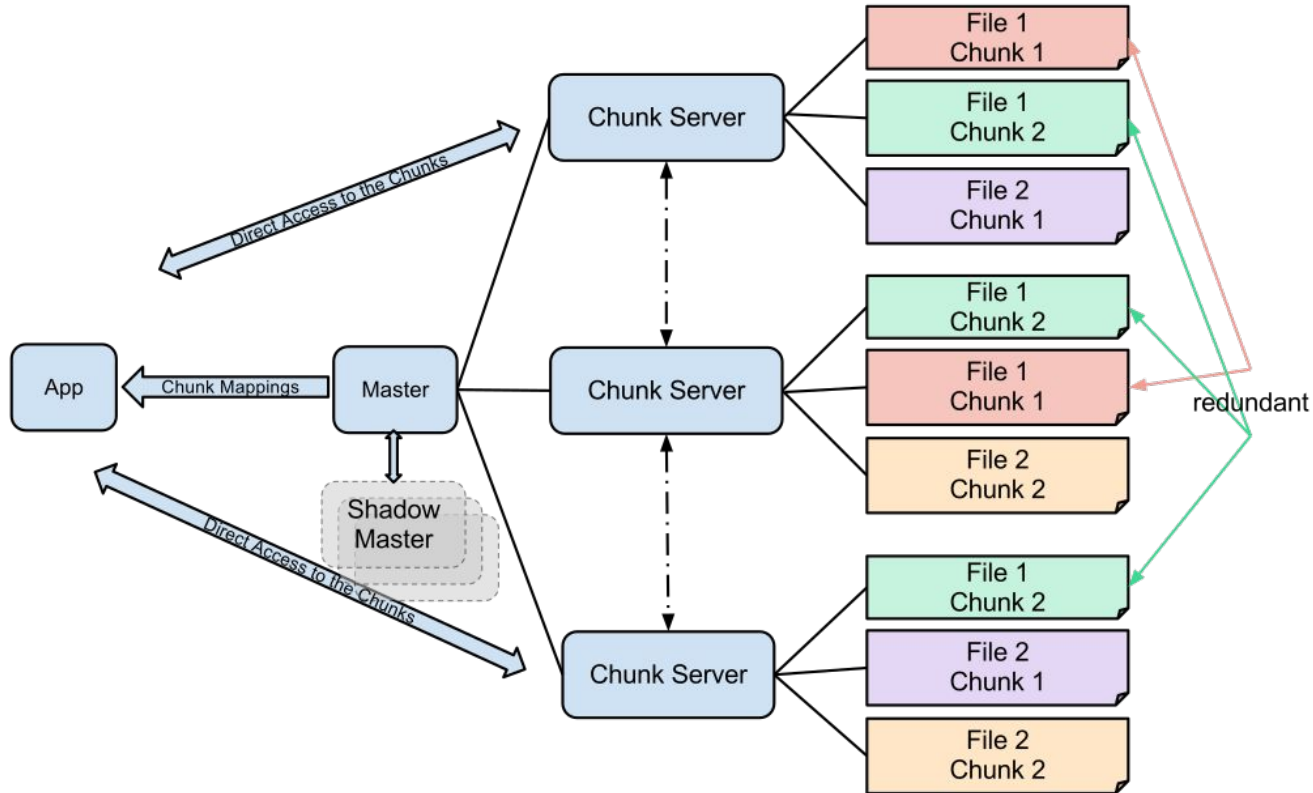
- Blocks of a huge file are stored on different machines
- We can process blocks locally on the machines where they are stored (fast read from local disk).
- If it is possible to process blocks independently in the task, then it is possible to achieve ideal scalability (embarrassingly parallel).
- For example, the task of filtering file lines scales perfectly. All lines are independently checked by the predicate, the more machines, the faster it will work.

MapReduce

girafe
ai

02

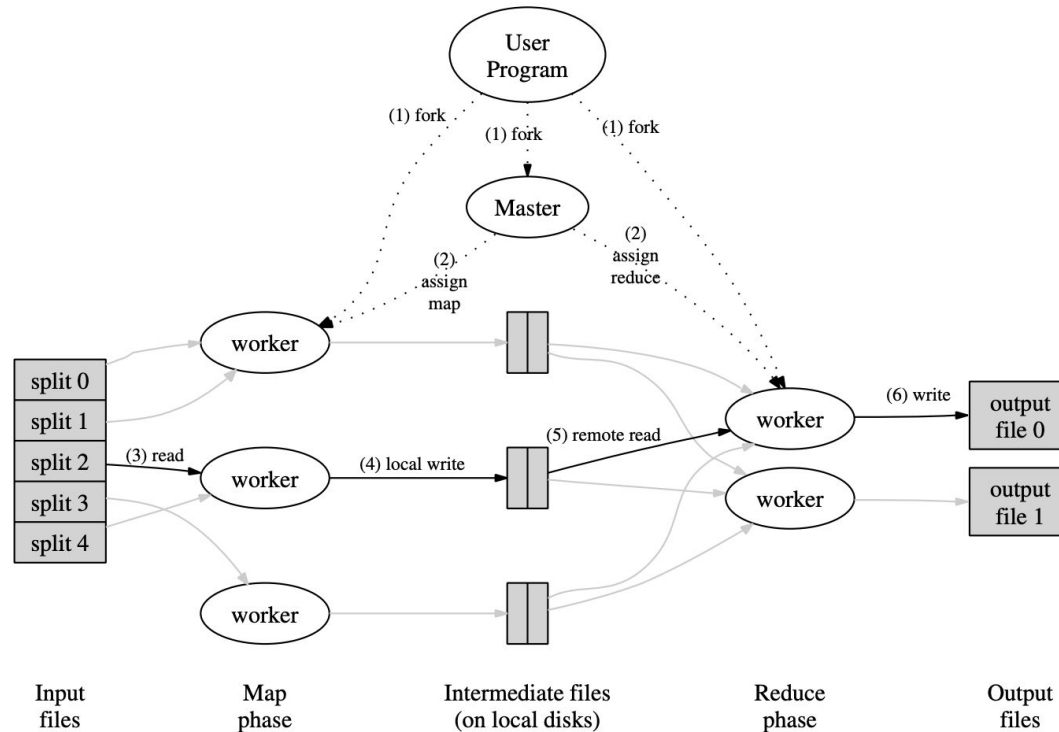
HDFS



GFS

<https://pdos.csail.mit.edu/6.824/papers/gfs.pdf>

MapReduce article



<https://static.googleusercontent.com/media/research.google.com/en/archive/mapreduce-osdi04.pdf>



Word frequencies counting problem

Texts from all over the Internet are stored in a distributed system.

We are interested to know the frequencies of all words (counters for tf-idf).

Solution is MapReduce paradigm

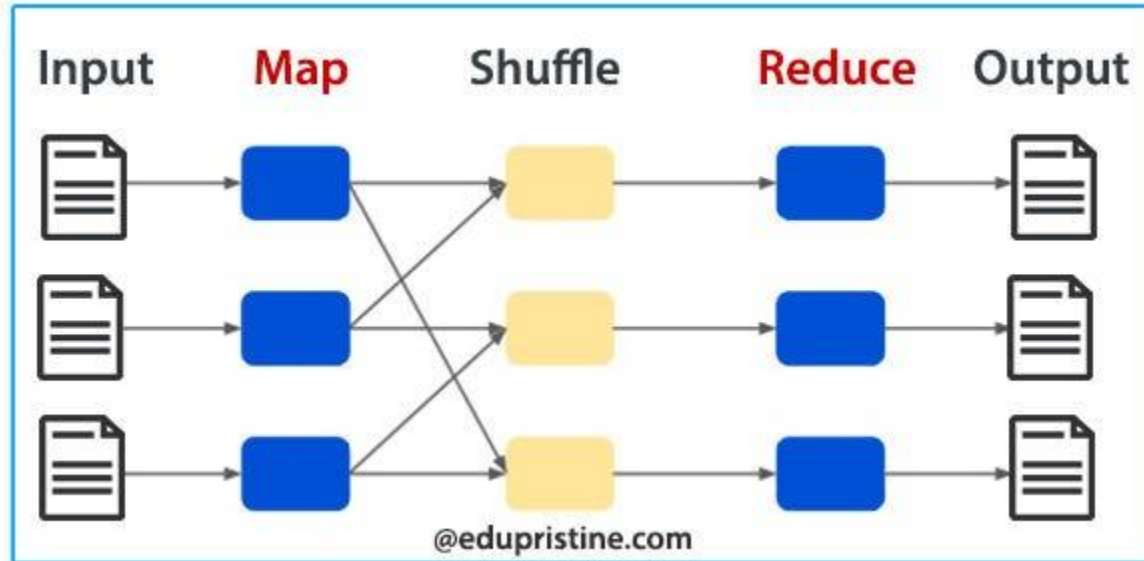
How to solve:

- for each block, we calculate the frequency of words in it (ideally scaled)
- add up the frequencies for all blocks

Big data course at MIPT: [2024](#), [2025](#)



MapReduce main idea





MapReduce solution

Map:

$(K1, V1) \Rightarrow \text{List}(K2, V2)$

(line number, "cat sleeps") $\Rightarrow$ [("cat", 1), ("sleep", 1)]

Shuffle:

Keys are splitted by $\text{hash}(\text{key}) \% N$ on N parts

Each parts gets sorted by **key** (independently)

Reduce:

$(K2, \text{List}(V2)) \Rightarrow \text{List}(K3, V3)$

("cat", (1, 1, 1)) $\Rightarrow$ [("cat", 3)]

Reliability of operations



- If the mapper is lost, we can restart the task only for its blocks
- If the reducer is lost, we re-collect data from all mappers only for its hash value



Popular track example

UserId	TrackId	AlbumId
11123	4521	842
14322	3593	957
...	...	...

Map: $(K1, V1) \Rightarrow \text{List}(K2, V2)$

Reduce: $(K2, \text{List}(V2)) \Rightarrow \text{List}(K3, V3)$

Let's take the logs of listening to music.

How do we find the most popular track for each album?

M: $\#, (\text{user}, \text{track}, \text{album}) \Rightarrow (\text{album}, \text{track}), 1$

R: $(\text{album}, \text{track}), (1, 1, \dots) \Rightarrow (\text{album}, \text{track}), \text{count}$

M: $(\text{album}, \text{track}), \text{count} \Rightarrow \text{album}, (\text{track}, \text{count})$

R: $\text{album}, \text{tracks} \Rightarrow \text{album}, \text{most popular track}$

Example: Item-based CF concurrency

**girafe
ai**

03

Example: Item-based CF concurrency

- n users, m items, $n \sim 10^6$, $m \sim 10^7$
- It is necessary to calculate in advance the Item-Item similarity

$$s(i, j) = \frac{\sum_{u \in U} (r_{ui} - r_u)(r_{uj} - r_u)}{\sqrt{\sum_{u \in U} (r_{ui} - r_u)^2} \sqrt{\sum_{u \in U} (r_{uj} - r_u)^2}} \quad \begin{array}{l} \text{adjusted} \\ \text{cosine} \\ \text{similarity} \end{array}$$

- Naive approach – $O(m^2n)$
- The evaluation matrix is very sparse, can it be better?
- The contribution to the similarity of the two products is non-zero only from users who rated both of these products



Inverted index

Update the calculations once a day using the inverted index



	2		2	4	5	
	5		4			1
			5		2	
		1		5		4
			4			2
	4	5		1		



$$s(i, j) = \frac{\sum_{u \in U} (r_{ui} - r_u)(r_{uj} - r_u)}{\dots}$$

Map step:

$$(1, 3) \rightarrow (5 - 3.3) * (4 - 3.3)$$

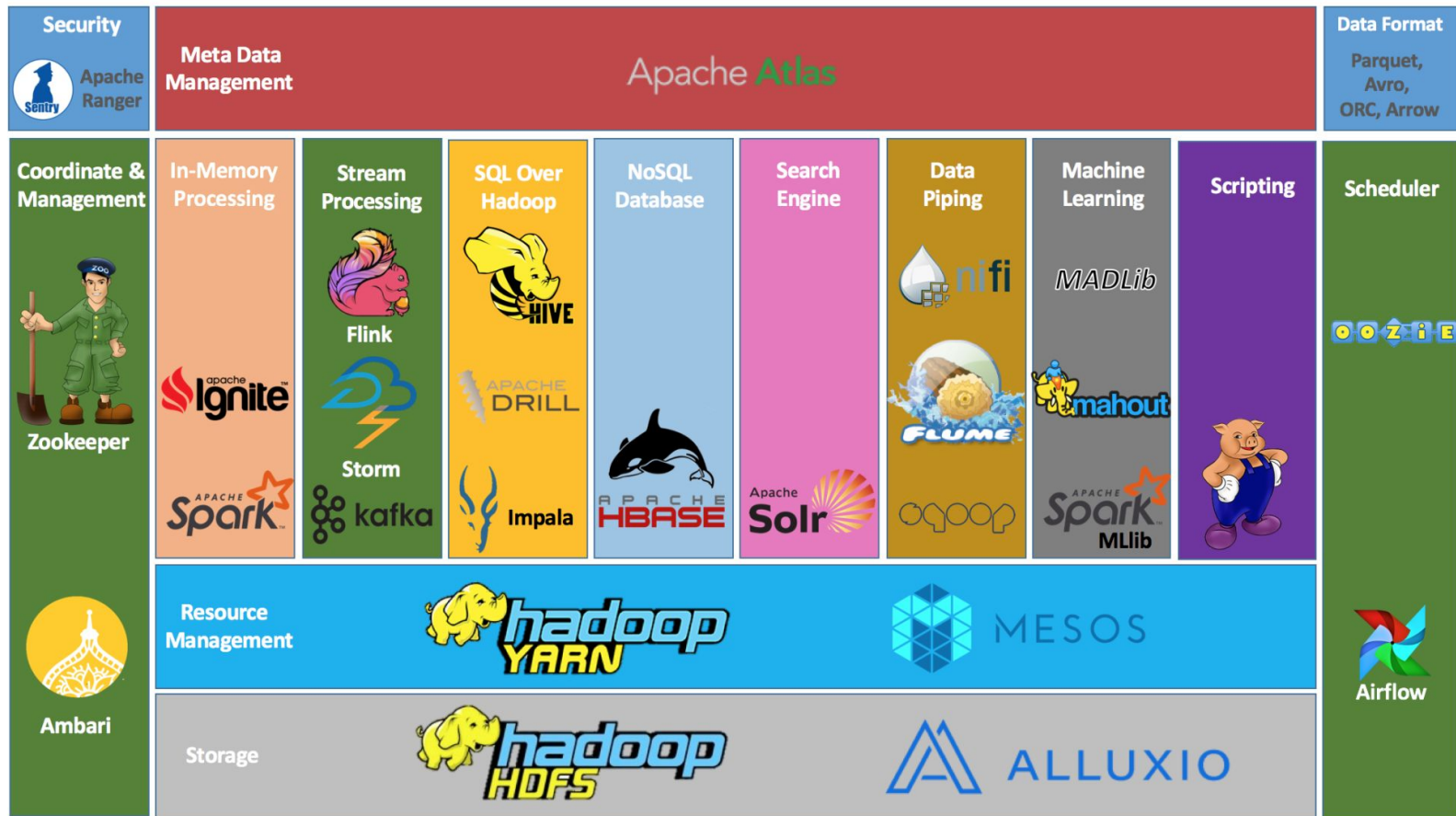
$$(1, 6) \rightarrow (5 - 3.3) * (1 - 3.3)$$

$$(3, 6) \rightarrow (4 - 3.3) * (1 - 3.3)$$

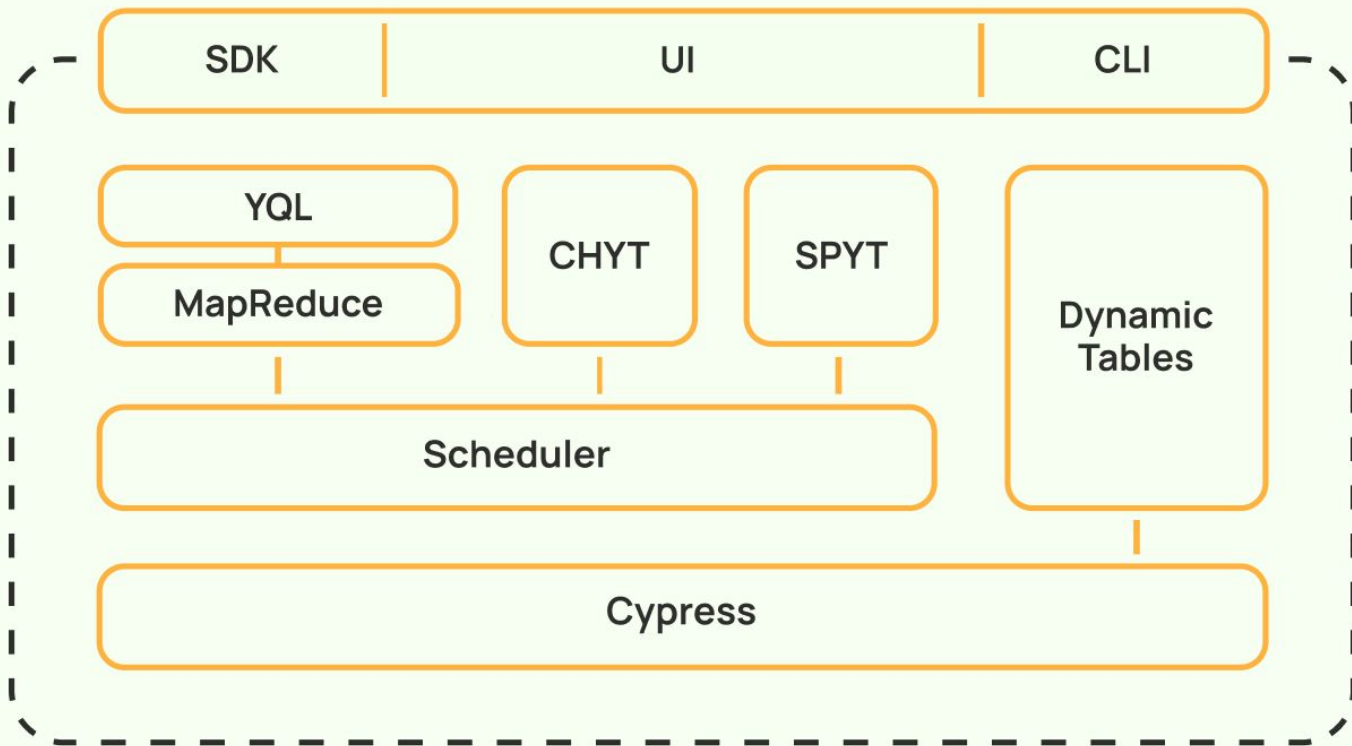
Reduce step: summing up by the key



Hadoop: open source MapReduce

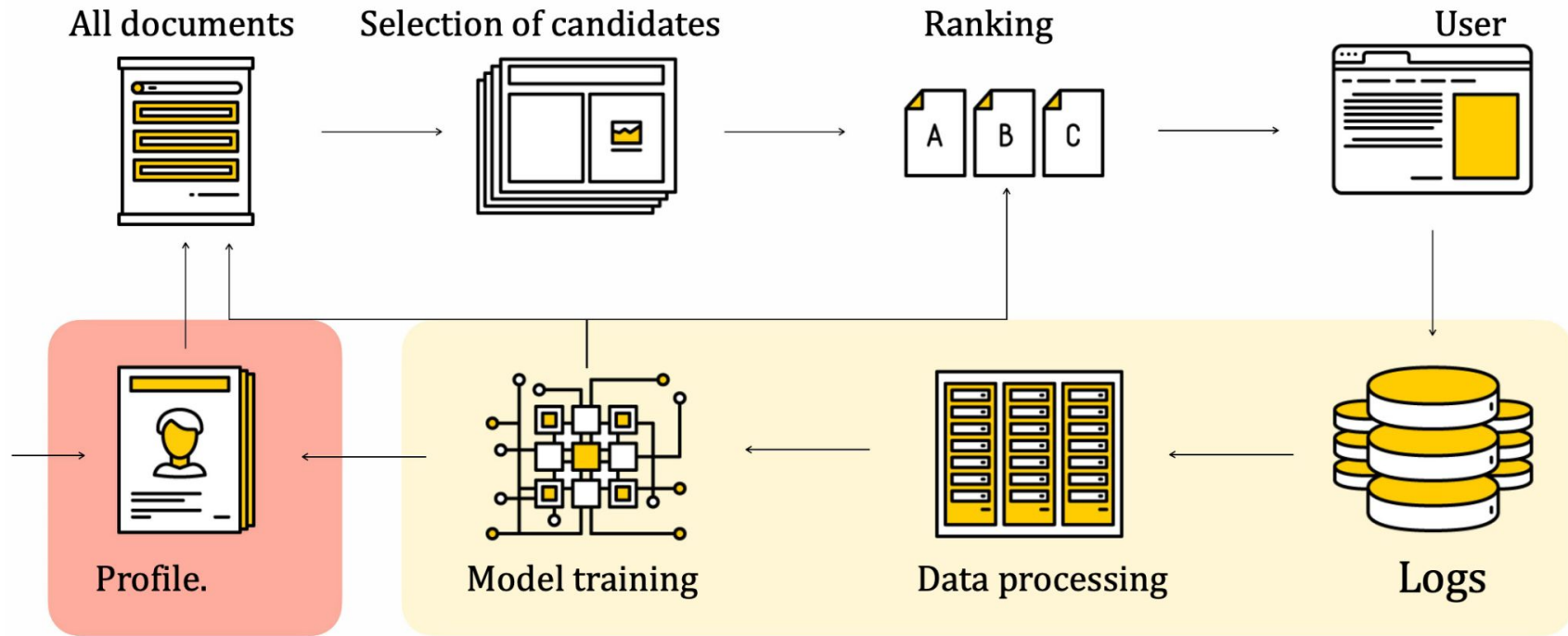


YTsauros: one more MapReduce





User Profile



Fast data storage

girafe
ai

05



Fast data storage

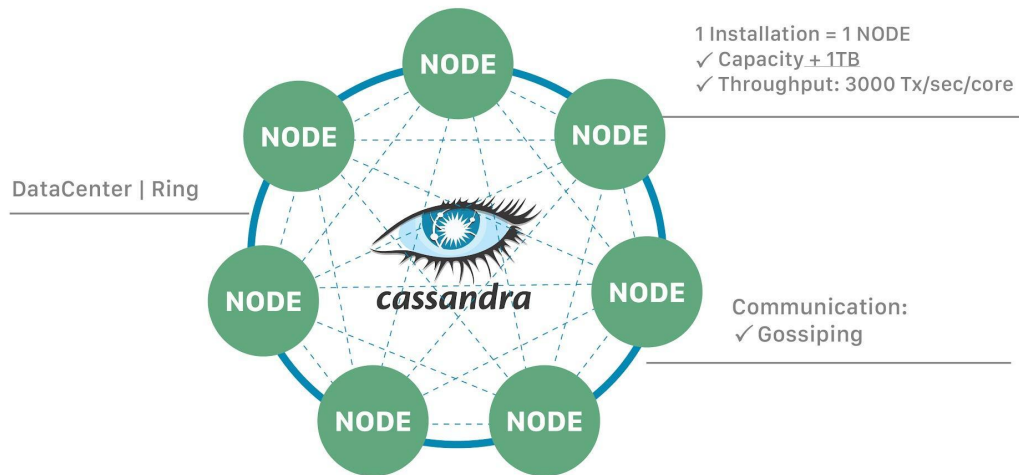
Key-value

- Reading and writing for $O(1)$
- Eventual consistency

Examples:

- Apache Cassandra
- Amazon DynamoDB
- Redis

ApacheCassandra™ = NoSQL Distributed Database



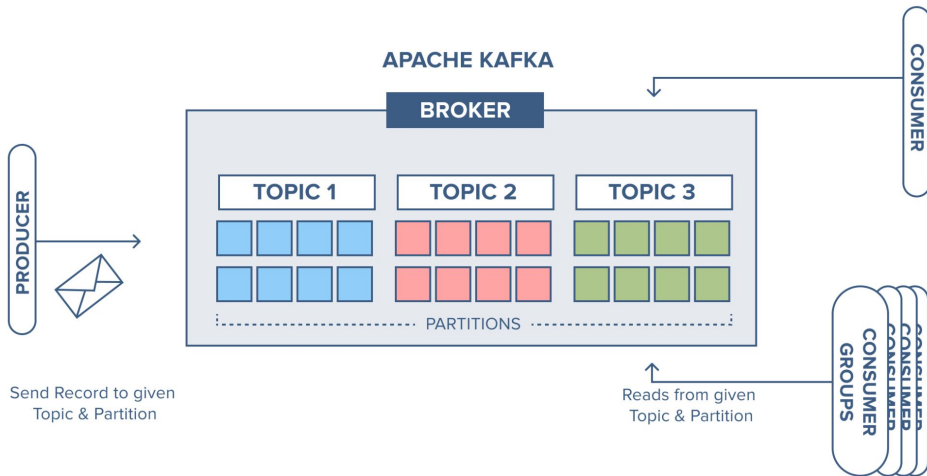


Near-realtime event handling

- We want to update user data as quickly as possible
- We update statistics, model results on the fly, reading updates from event queues

Example: writing user actions in Kafka

1. collect the user profile from the Key-Value response and events from Kafka
2. process statistics and do the ALS step every N minutes



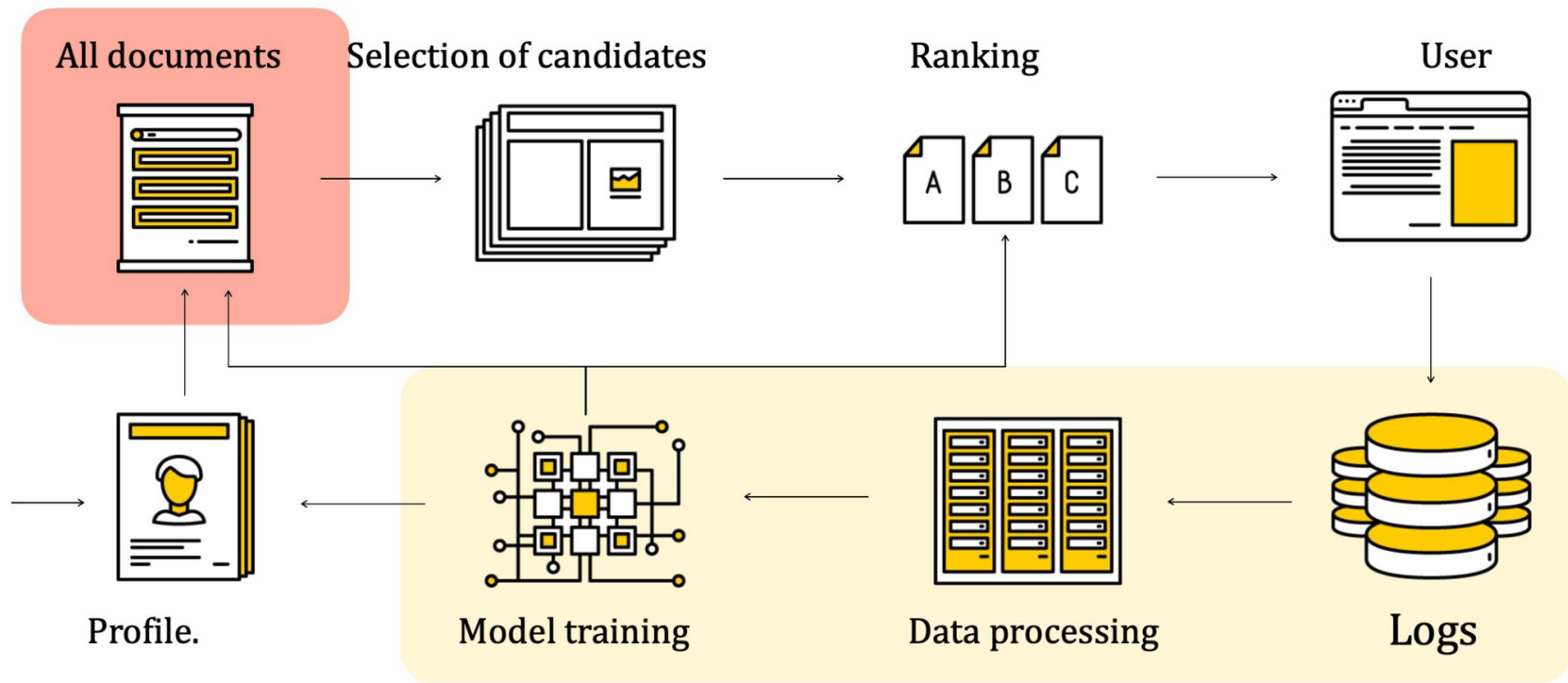
Document storage

girafe
ai

06



Document storage

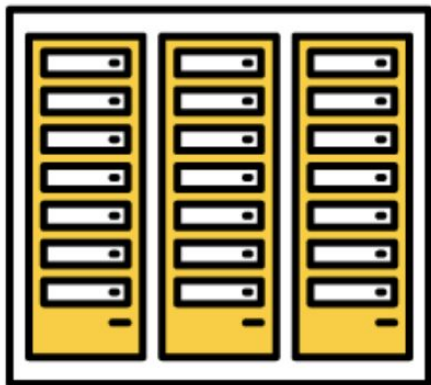


Sharding



- One server has a limit on the number of documents stored
- Let's store disjoint sets of documents on different servers, and combine the results of responses

One shard

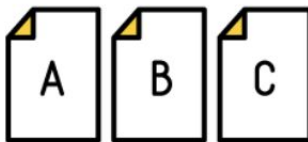
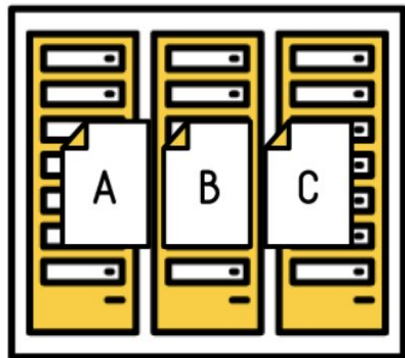


Sharding



- One server has a limit on the number of documents stored
- Let's store disjoint sets of documents on different servers, and combine the results of responses

One shard



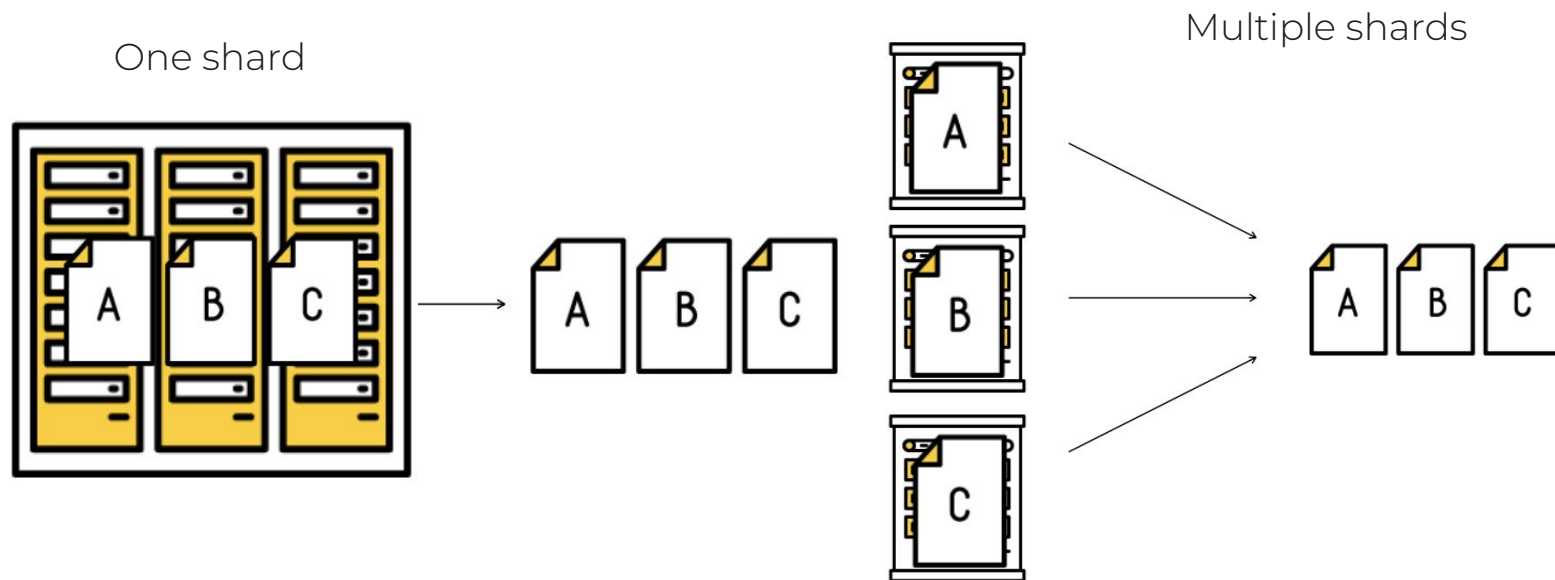
Multiple shards



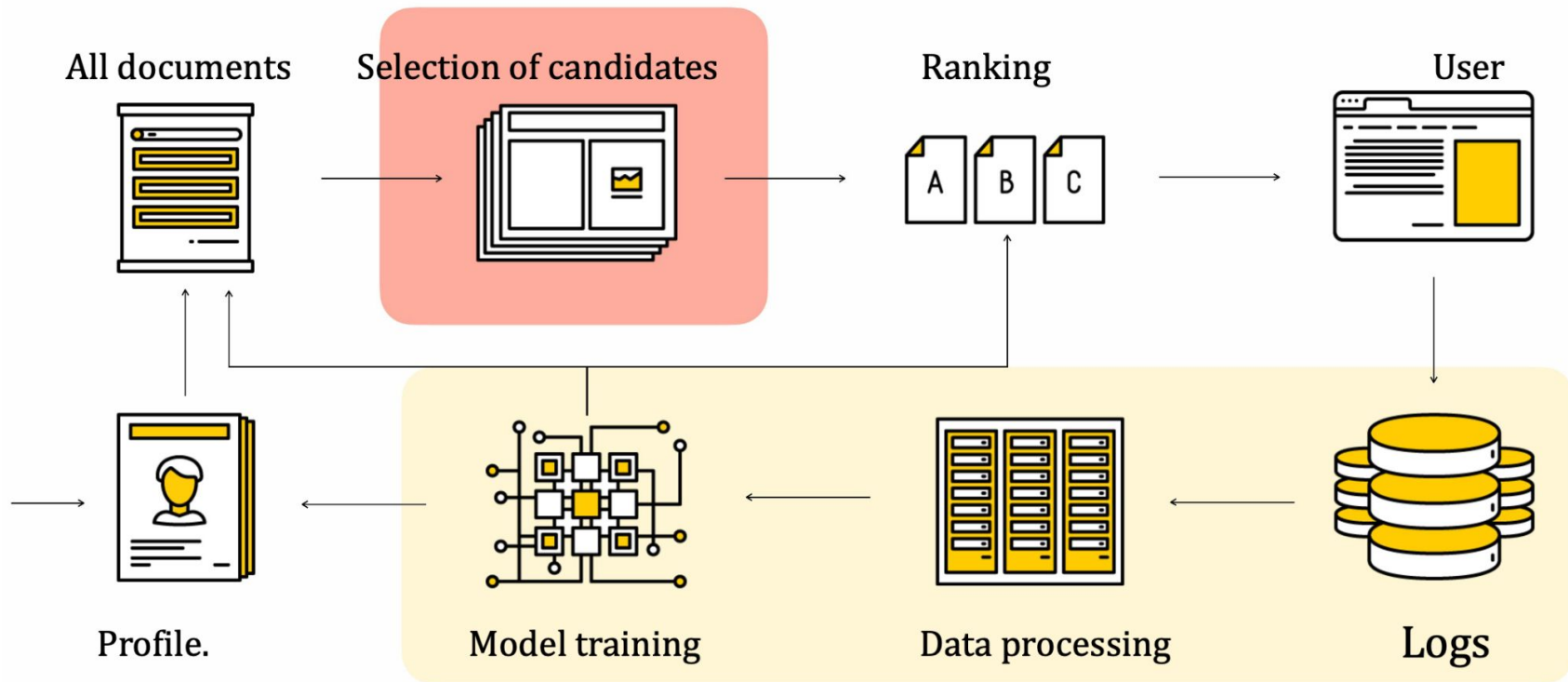
Sharding



- One server has a limit on the number of documents stored
- Let's store disjoint sets of documents on different servers, and combine the results of responses



Selection of candidates





Example: Item-based CF

- We will calculate for each product offline 1000 of the most similar to it, add it to the Key-Value storage (for example, Redis)
- Online, let's take the products evaluated by the user (the last 100) and go to Key-Value for similar ones for them

kNN indexes

girafe
ai

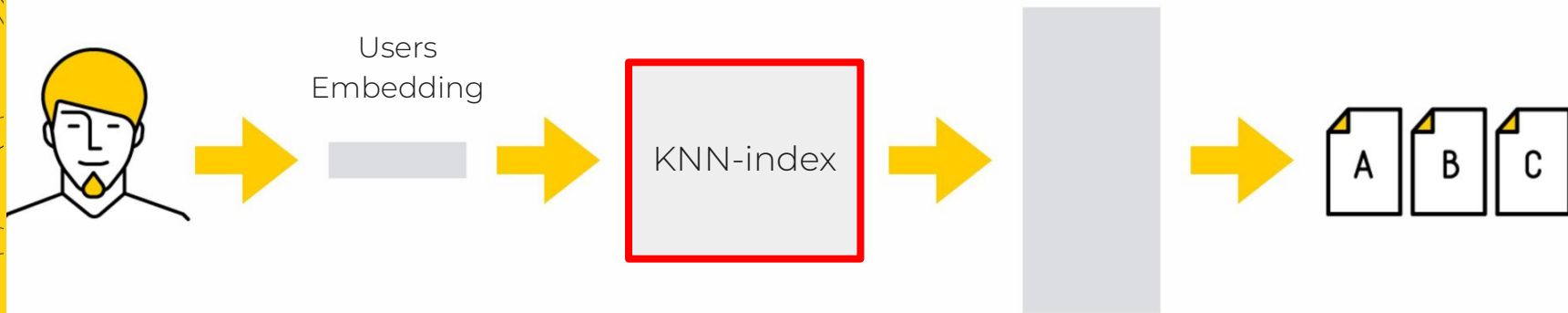
07



kNN index

KNN index is a data structure that allows you to quickly find the N nearest vectors for a given vector

Embeddings of the nearest documents



Ideas of kNN index



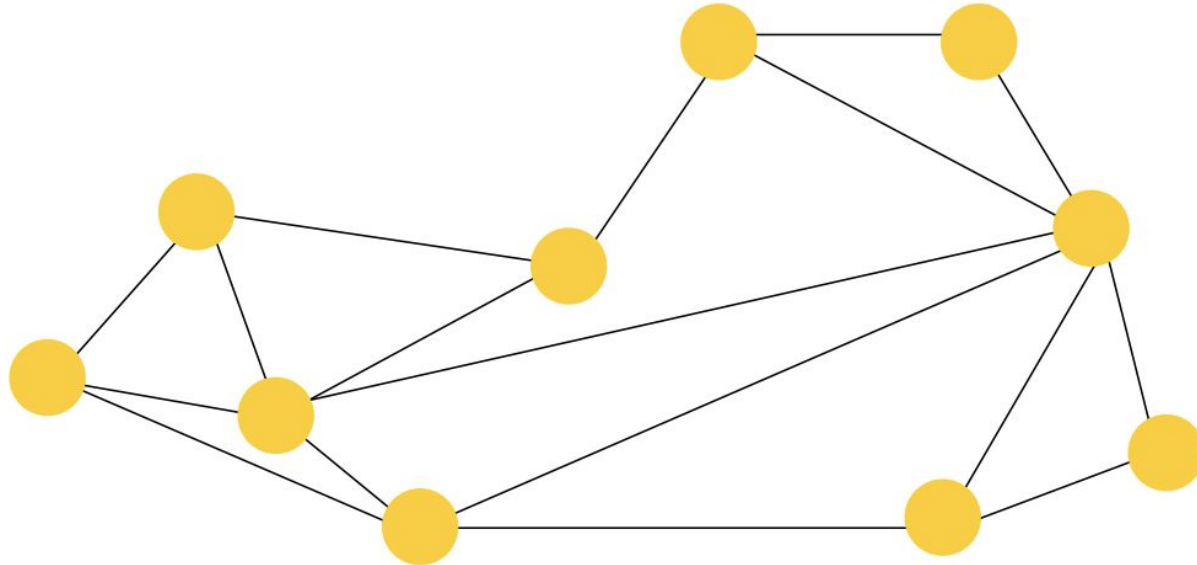
- [KD tree](#)
- [Ball tree](#)
- Clusters hierarchy (FAISS GPU)
- HNSW

Navigable Small World

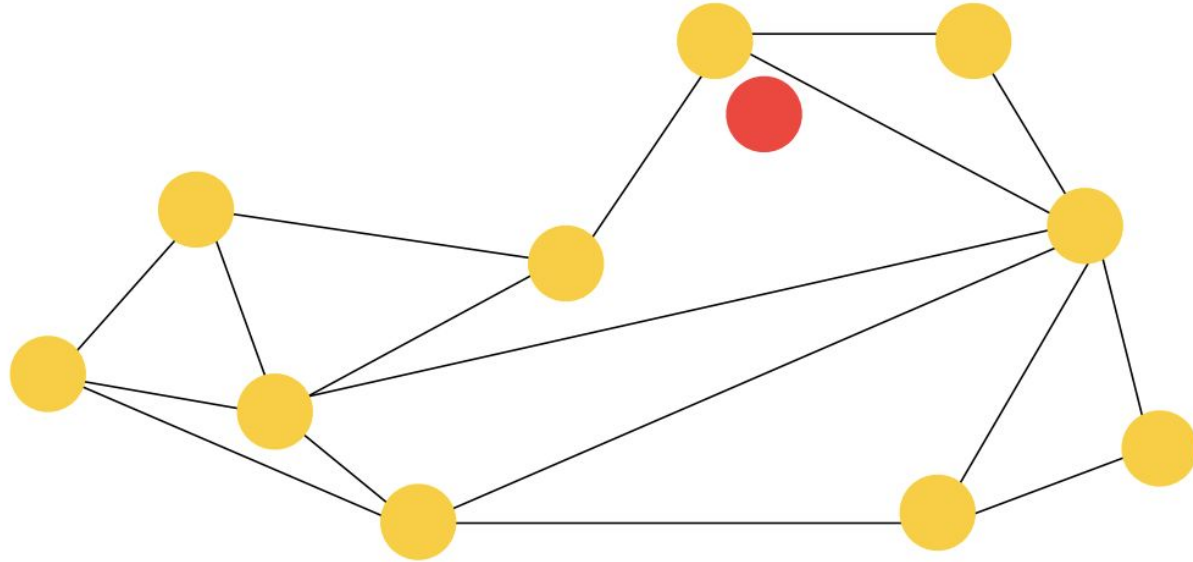


- A graph where vertices are embeddings, edges are distances
- Adding nearby and remote vertices as edges
- We start from a random vertex, count the distances and move along the graph closer to the query point

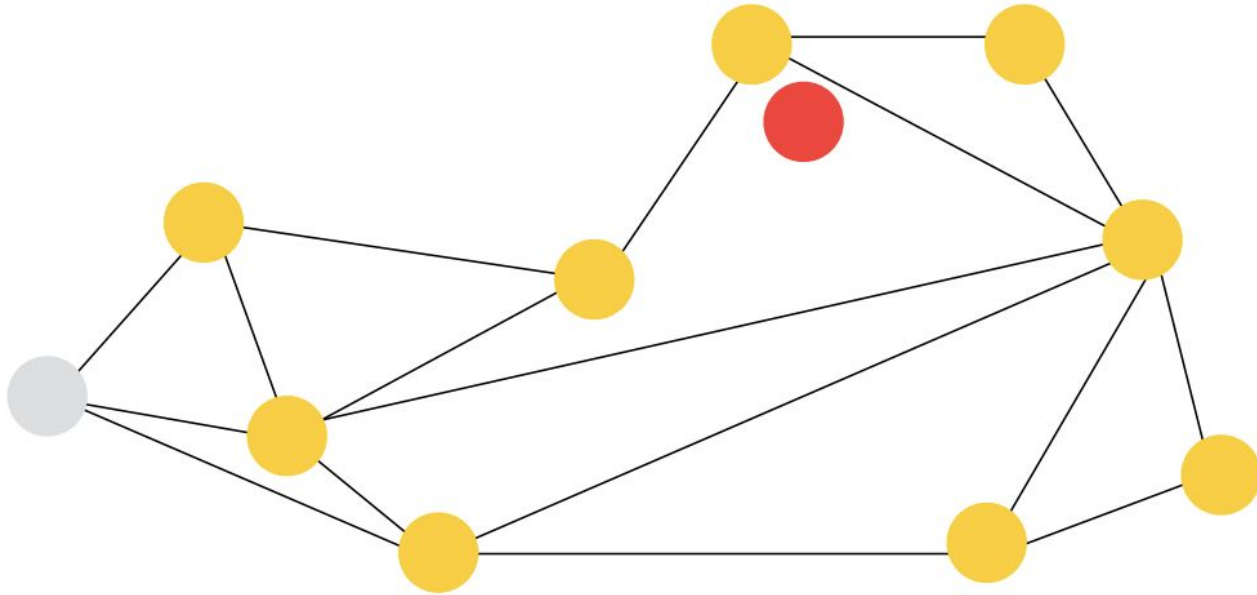
NSW (Navigable Small World)



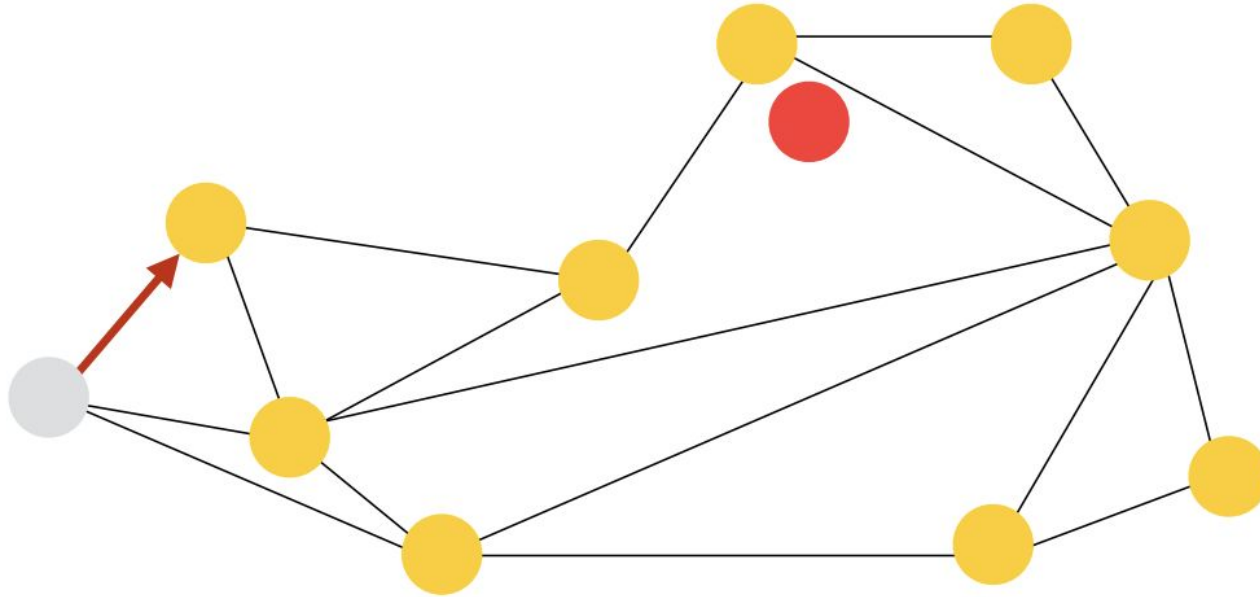
NSW (Navigable Small World)



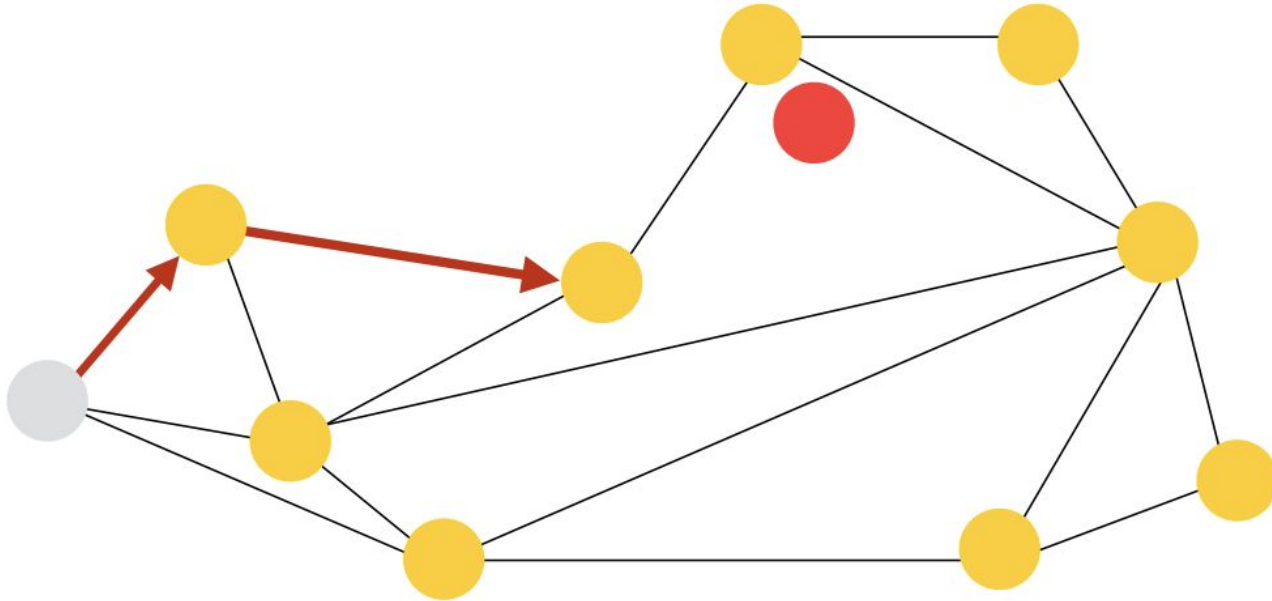
NSW (Navigable Small World)



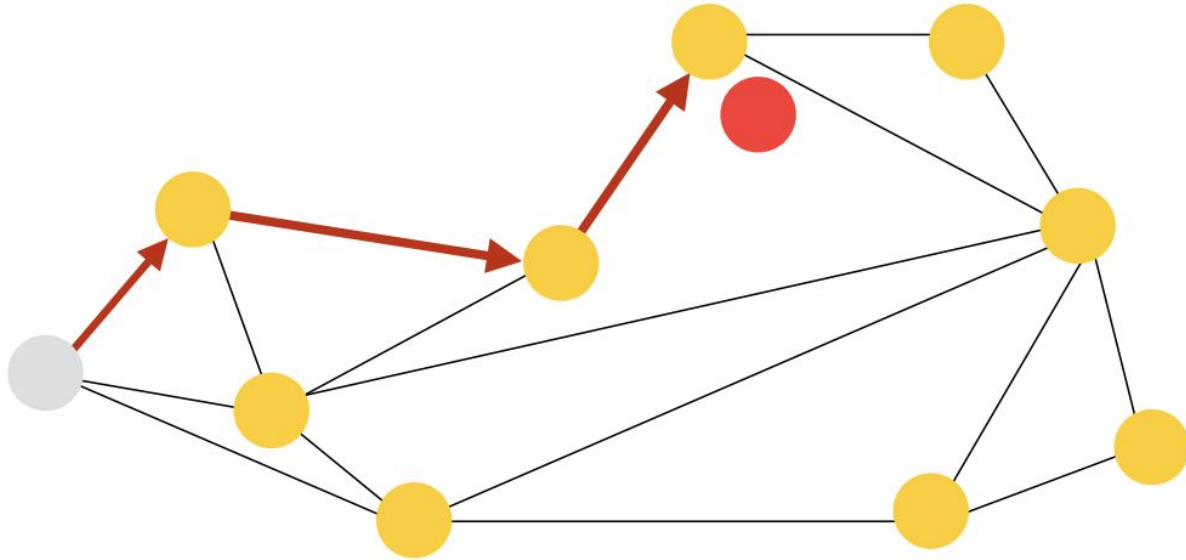
NSW (Navigable Small World)



NSW (Navigable Small World)



NSW (Navigable Small World)

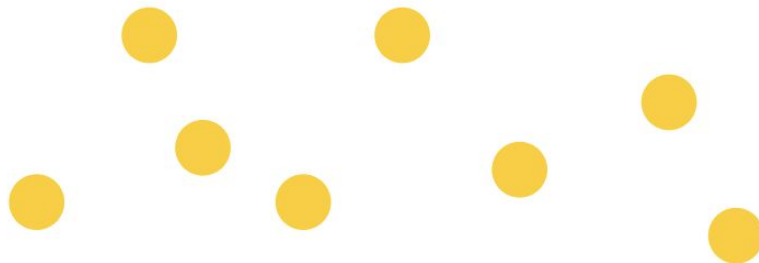


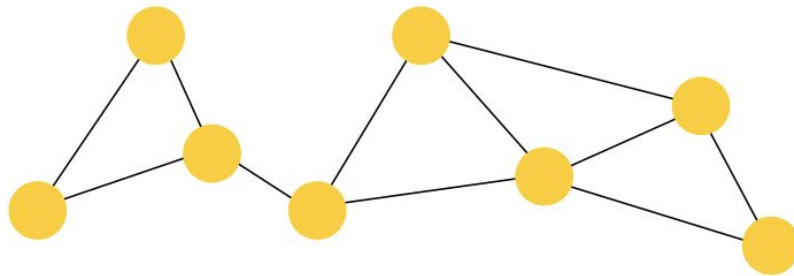
Hierarchical Navigable Small World (HNSW)

girafe
ai

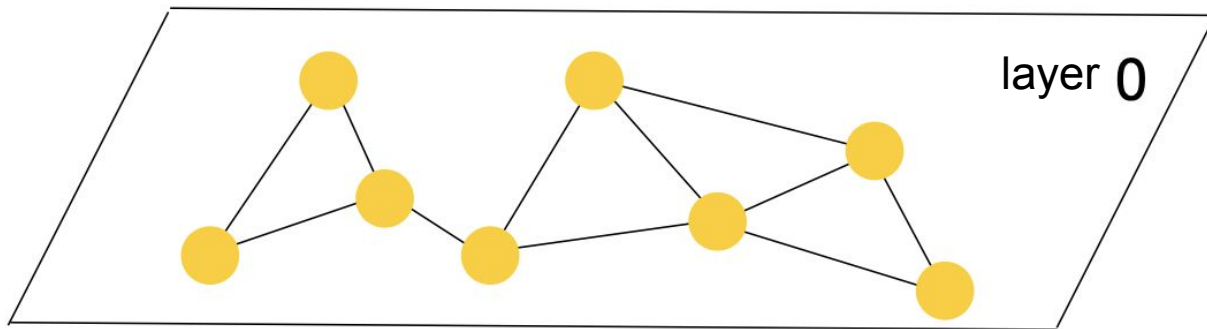
08

HNSW

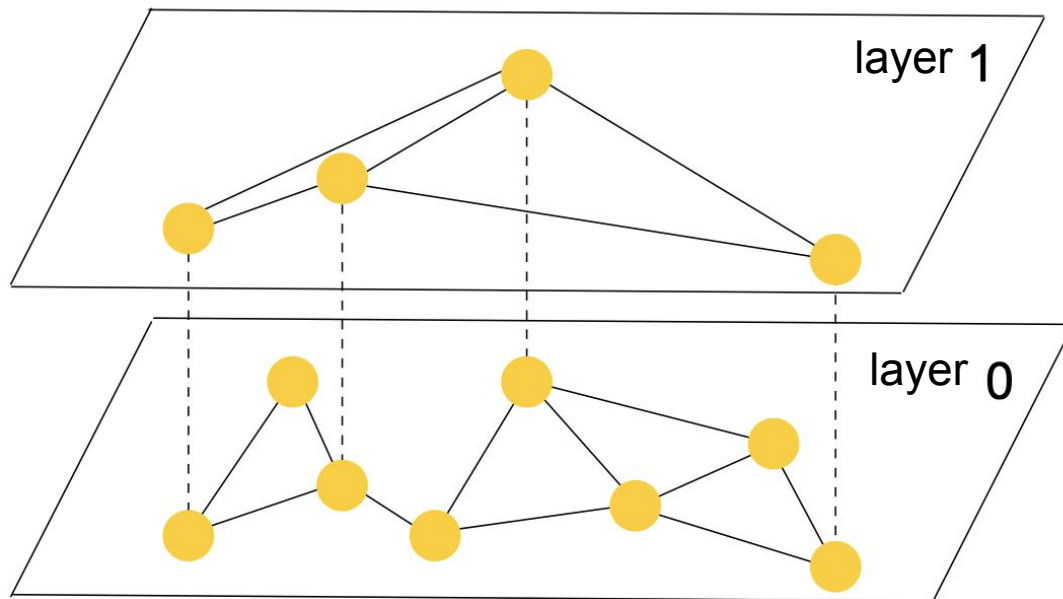




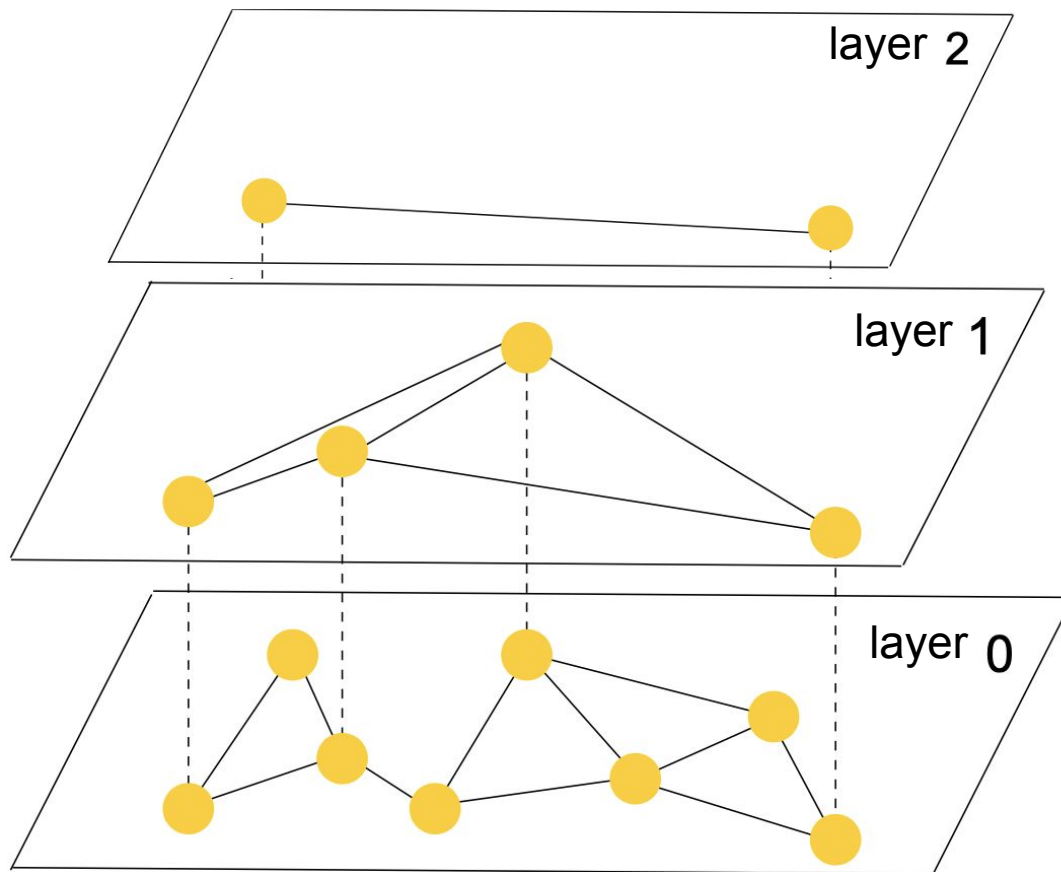
HNSW



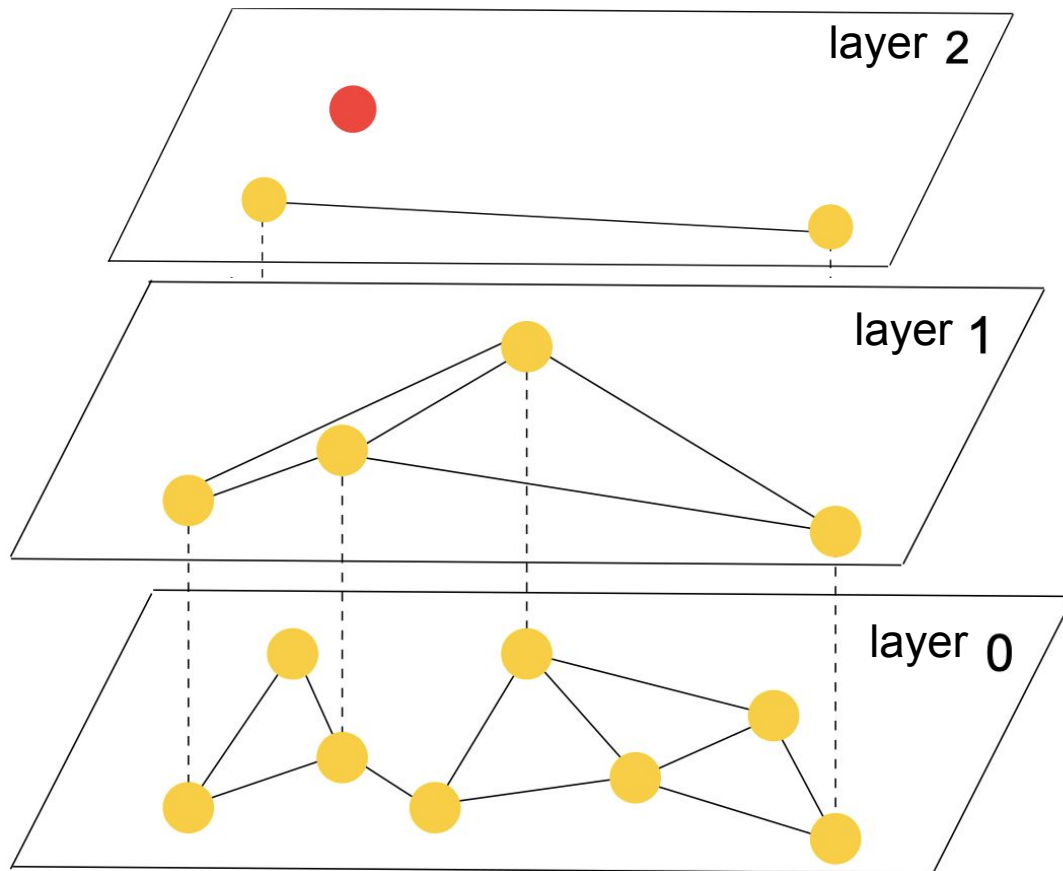
HNSW



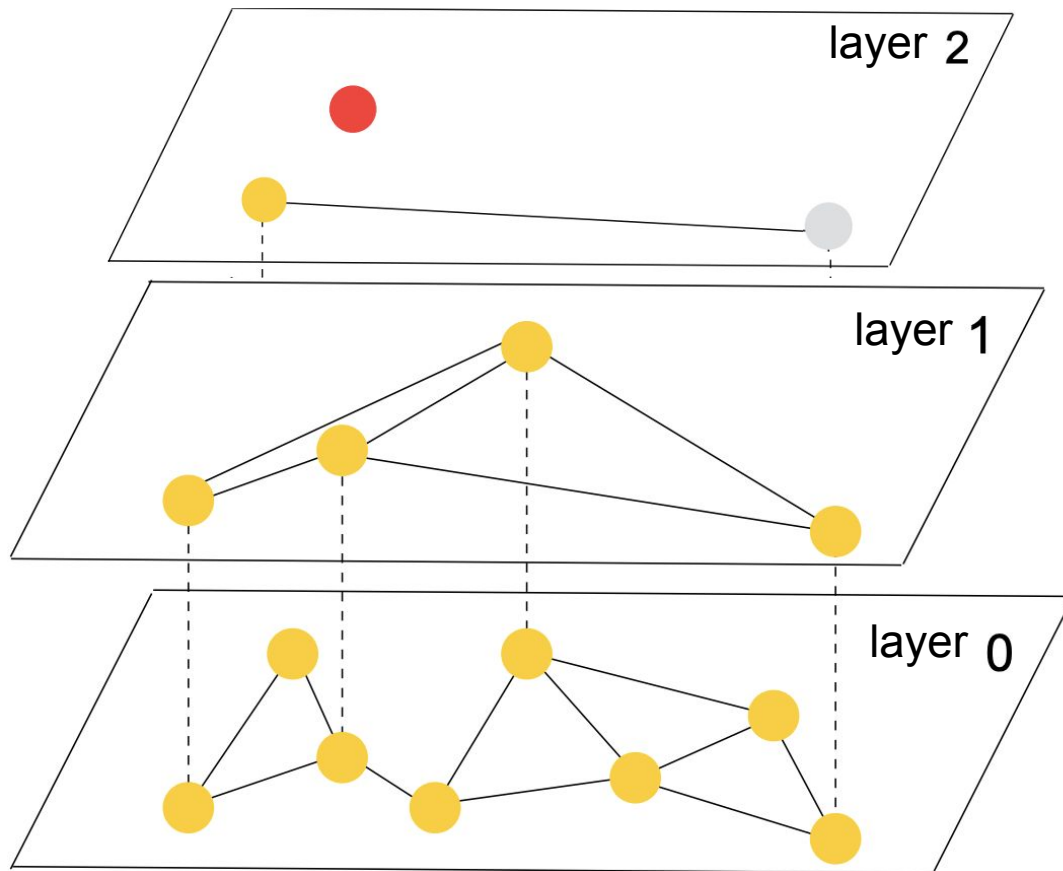
HNSW



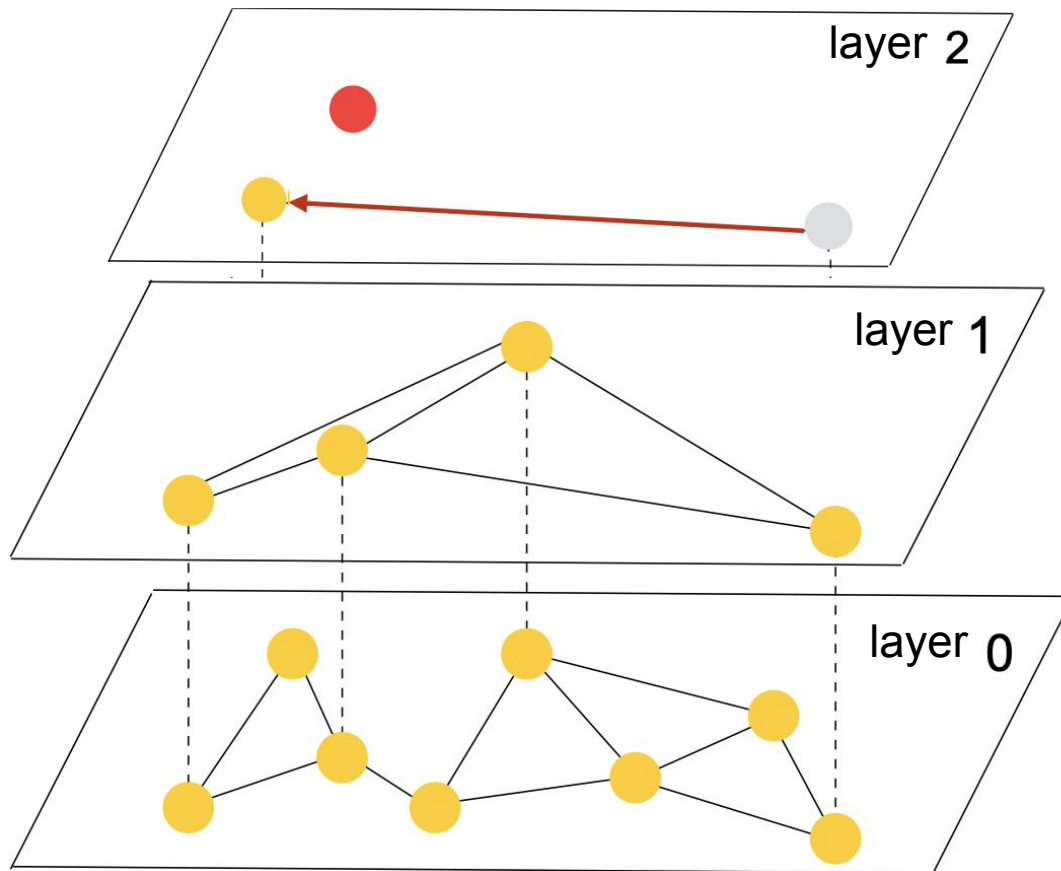
HNSW



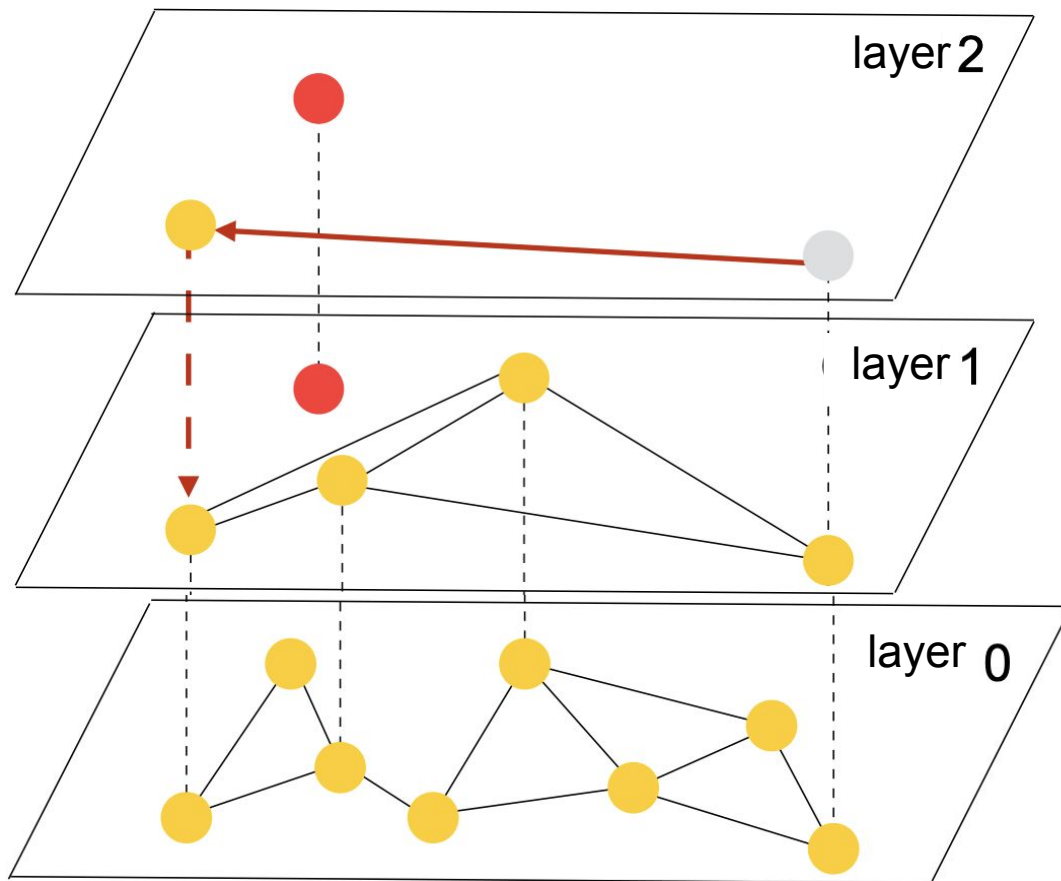
HNSW



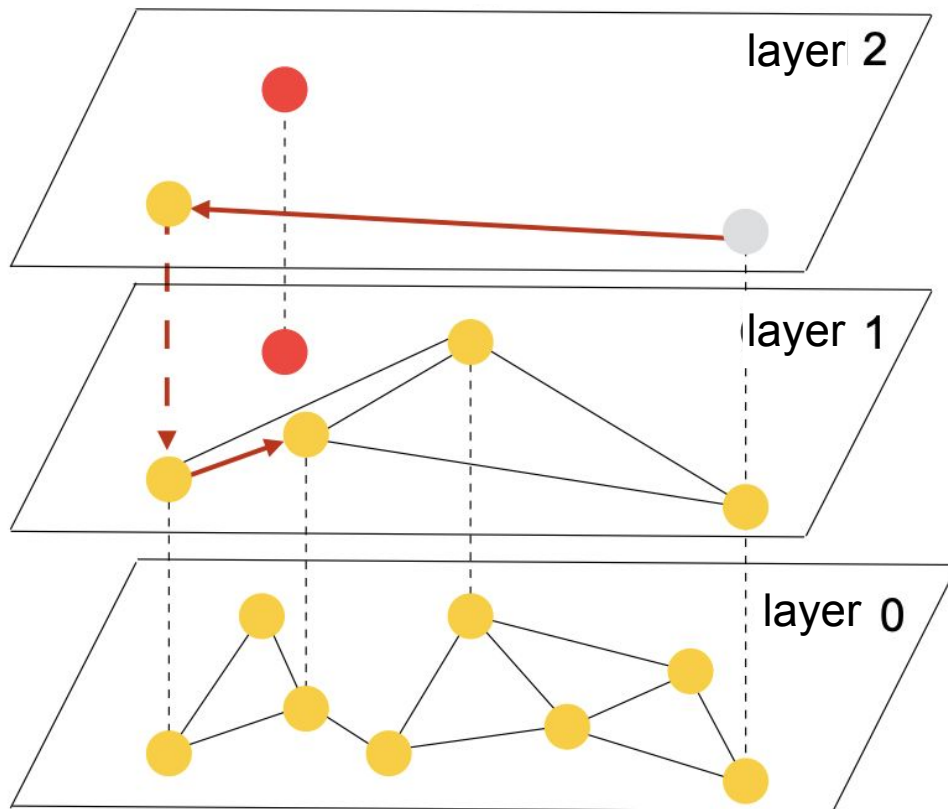
HNSW



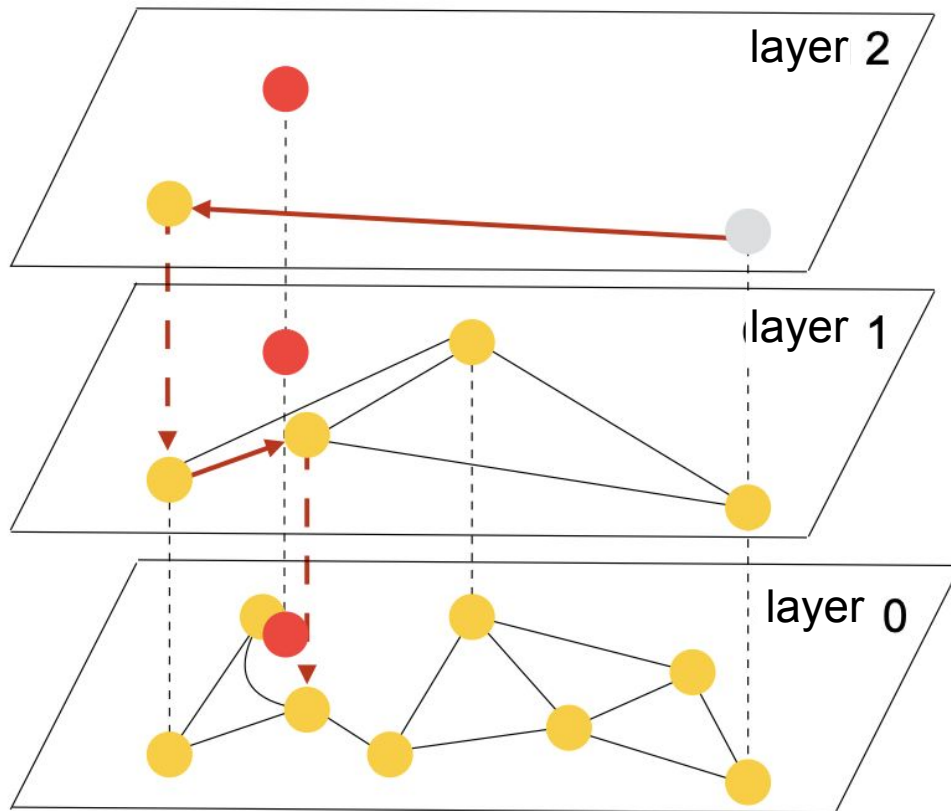
HNSW



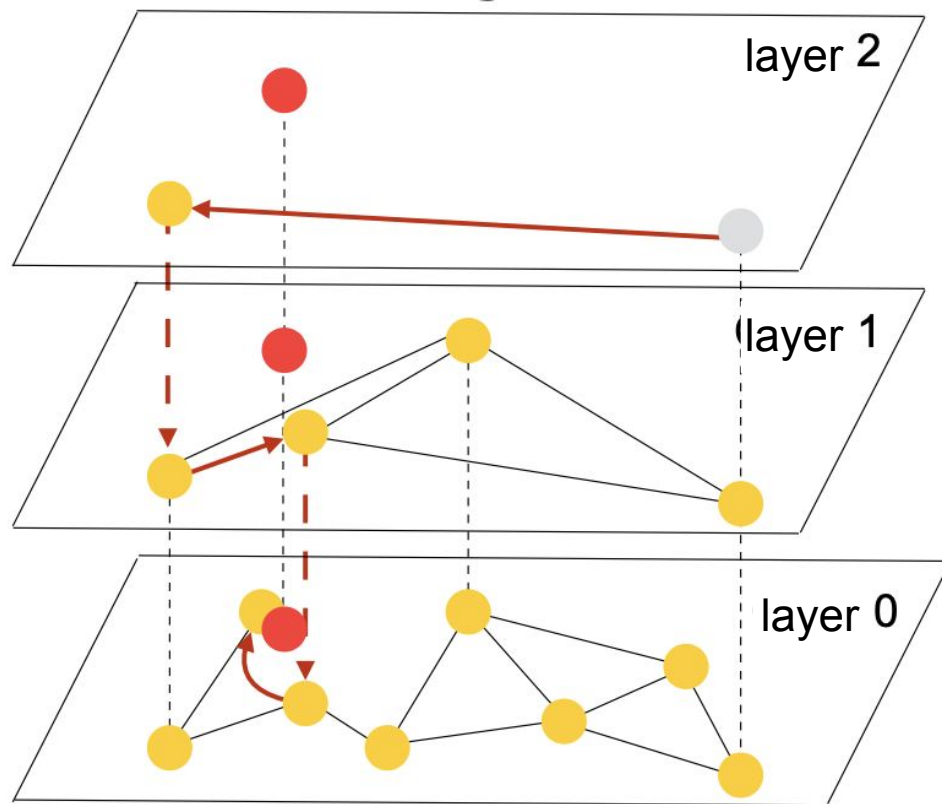
HNSW



HNSW



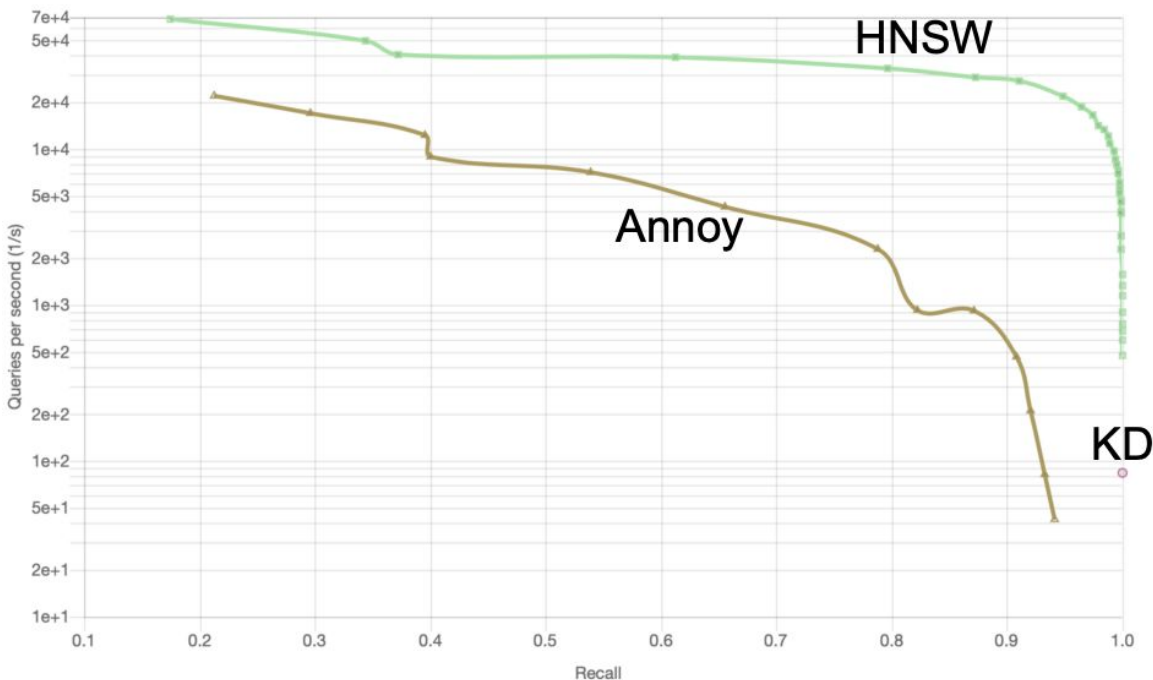
HNSW



Accuracy of approximate search



- In addition to speed, you need to find accurate nearby objects
- Comparison on ALS embedding last fm-64-dot, 360K





kNN index libraries

- <https://github.com/facebookresearch/faiss; wiki documentation>
- <https://github.com/nmslib/nmslib>
- <https://github.com/flann-lib/flann>
- <https://github.com/spotify/annoy>

Also list of vector databases from openAI

<https://platform.openai.com/docs/guides/embeddings/how-can-i-retrieve-k-nearest-embedding-vectors-quickly>

Services:

- <https://github.com/qdrant/qdrant>
- <https://feast.dev/> ?

Quantization

girafe
ai

09

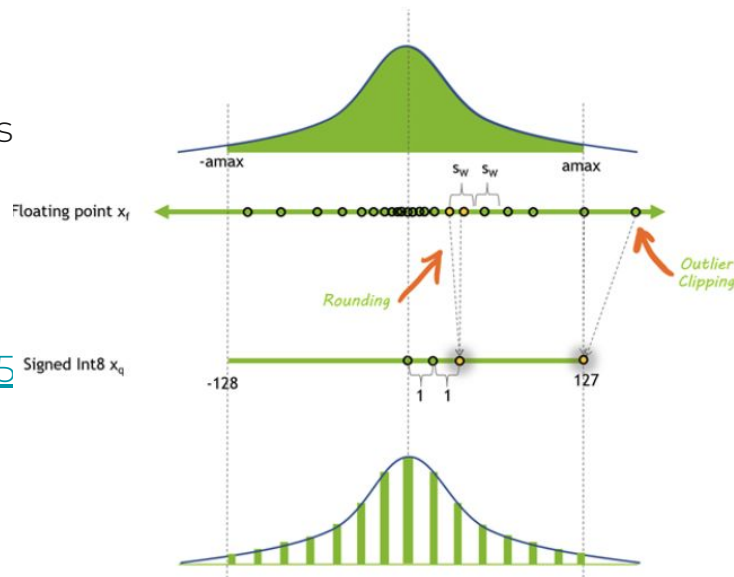
Quantization



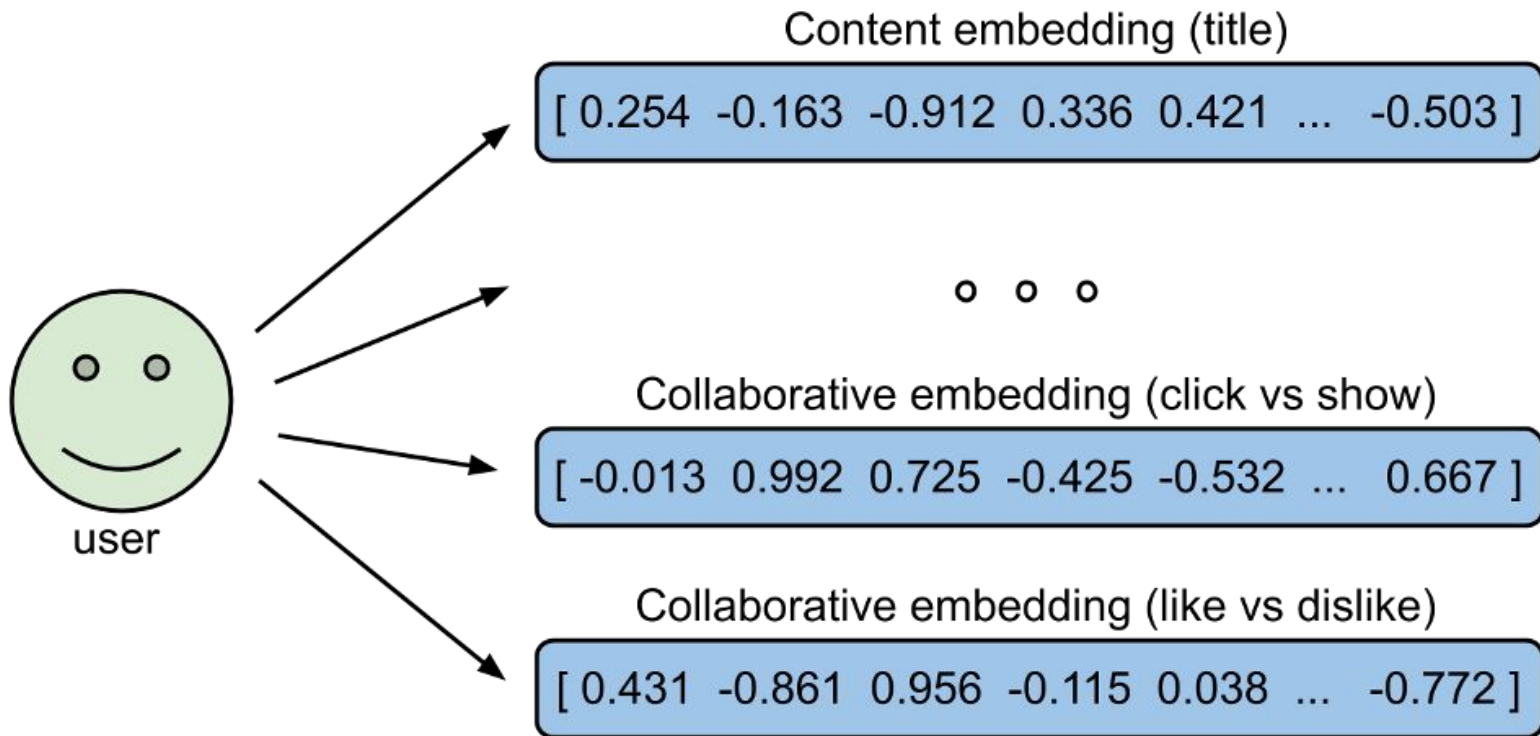
- Embedding elements — float32
- How much will a 100-dimensional vector take in bytes?
 - 400 bites
- And if you replace it with int8?
 - 100 bites
- So let's break down the values by quantiles

Dzen experience in quantization (ru)

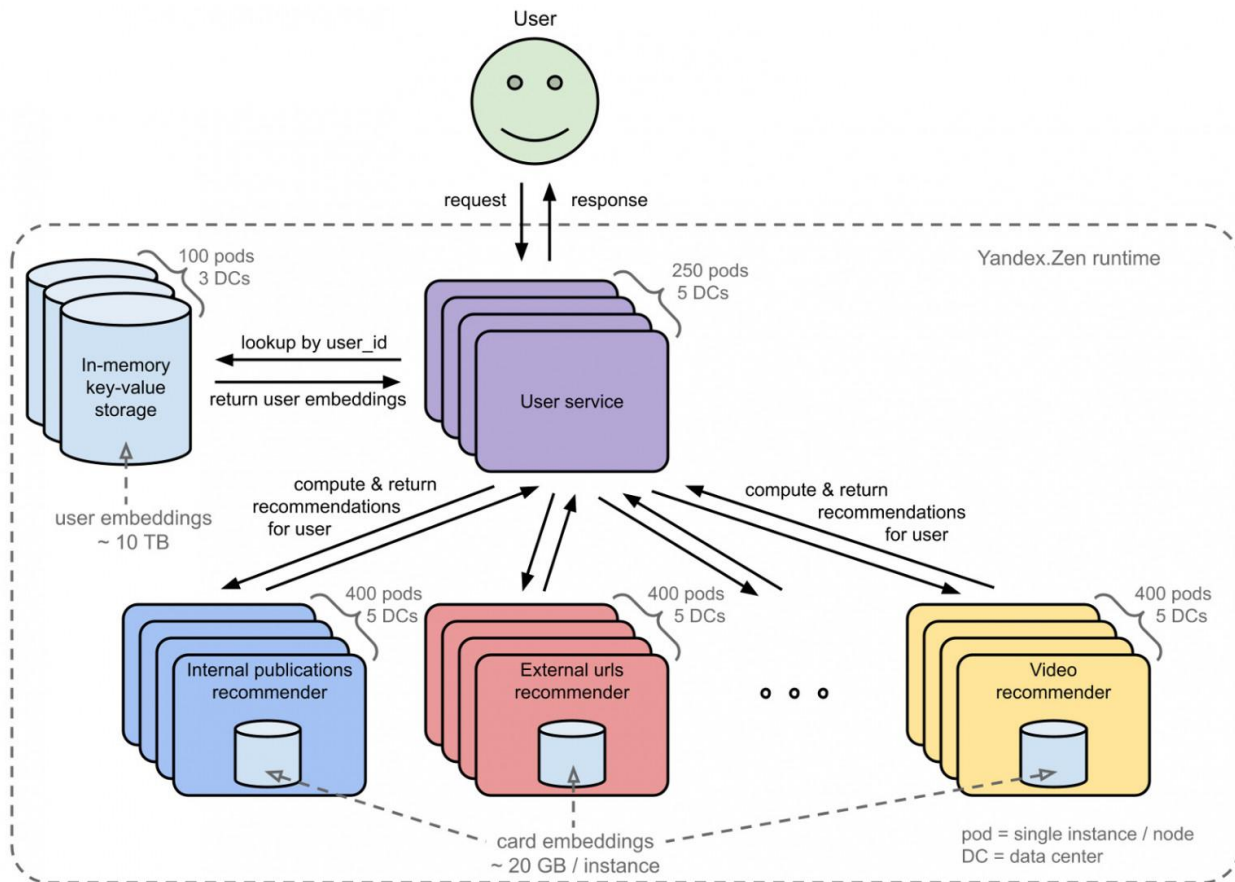
<https://habr.com/ru/companies/yandex/articles/5>



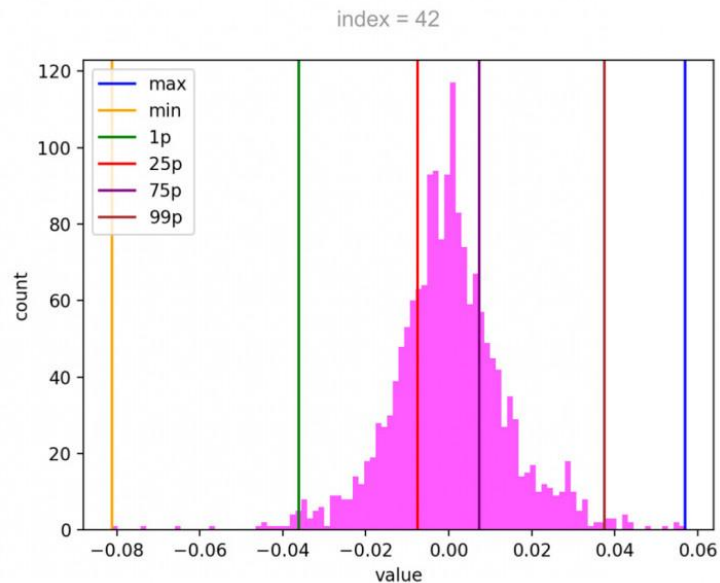
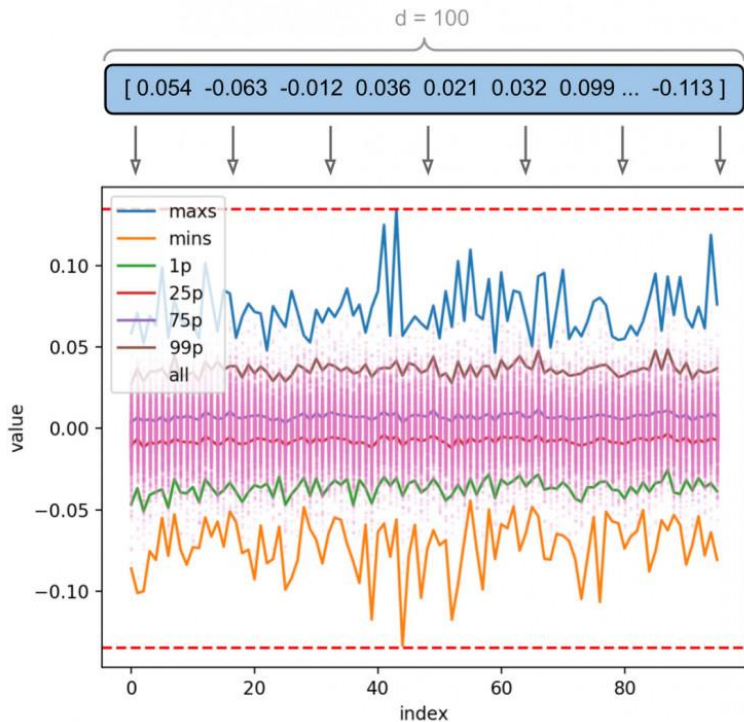
Quantization



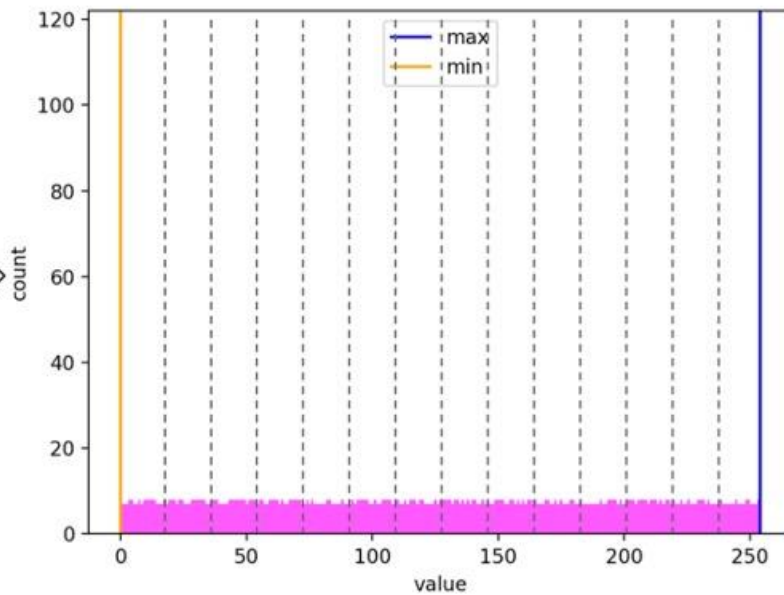
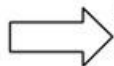
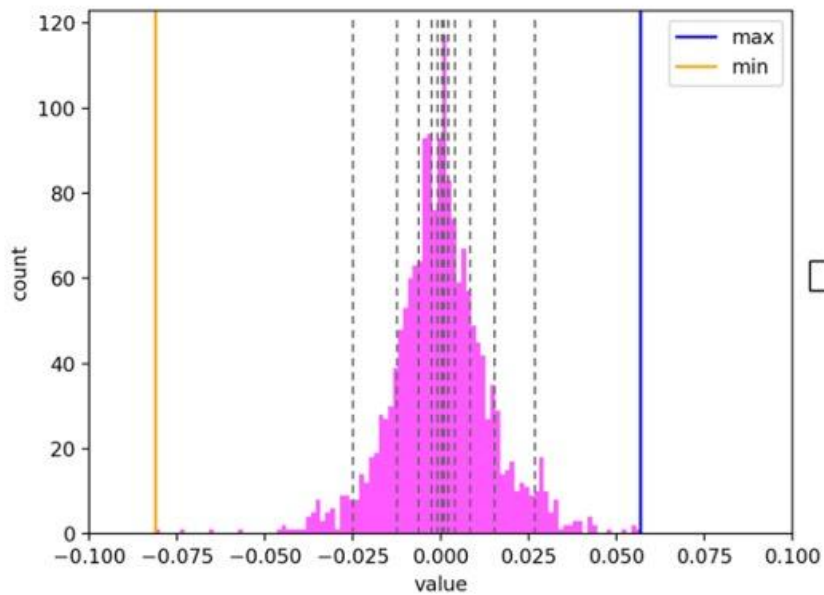
Quantization



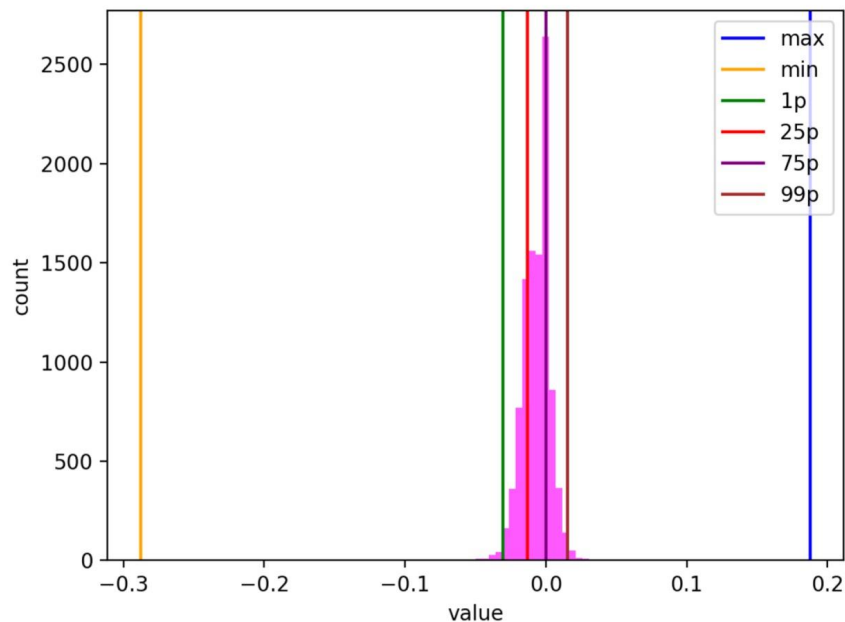
Quantization



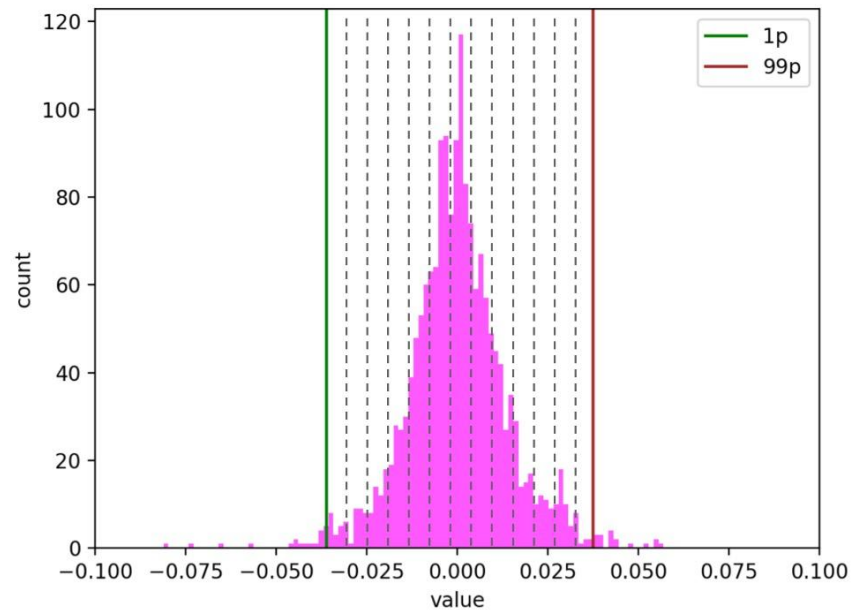
Quantization



Quantization



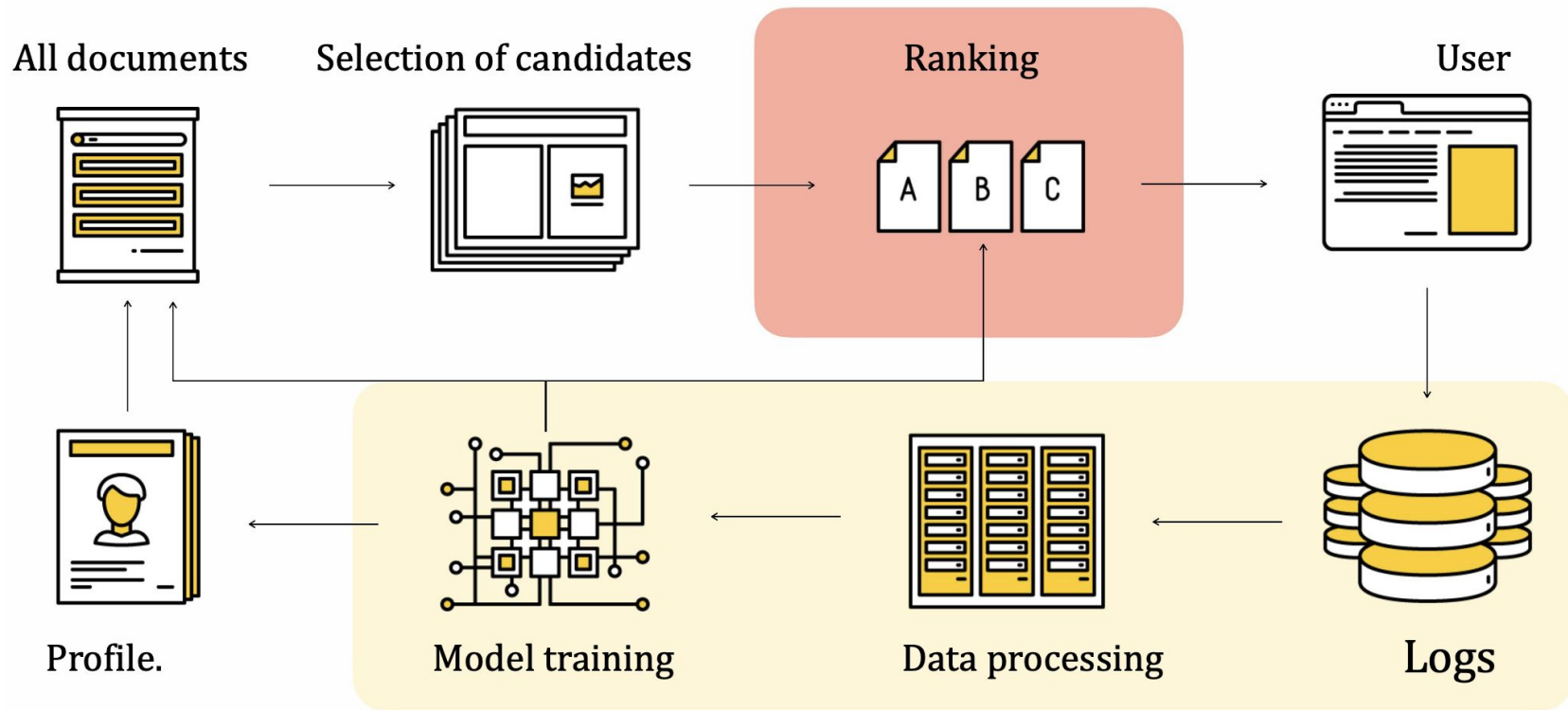
min-max distribution



1-99 percentiles distribution



Ranking



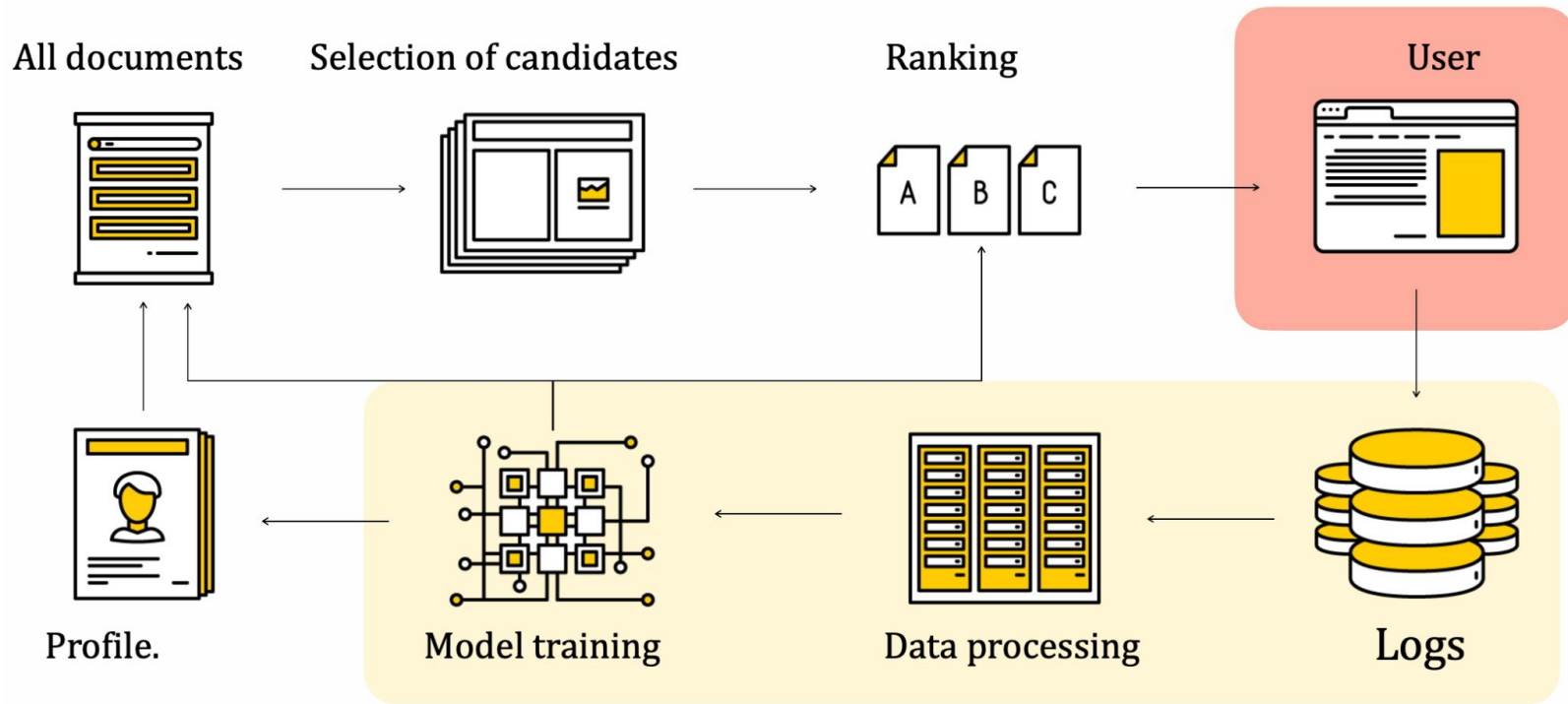
Ranking

In practice, several ranking models can be used, each of which is applied to the top of the documents selected by the previous one; gradient boosting is most often used as models. And almost always the response of the model is transformed using business rules: diversity, history of interaction with the document, freshness, time of day, etc.





Load from users



Caching during serving



- We get the results from the cache, reduce the load on the main service

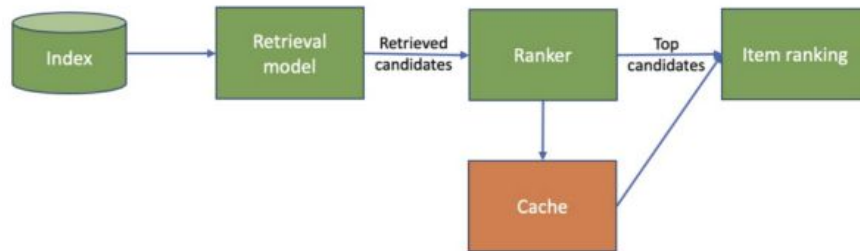


Figure 1: A Delivery Flow of Recommendation Systems with Simple Caching Strategy

- Adding a model that predicts cache usage

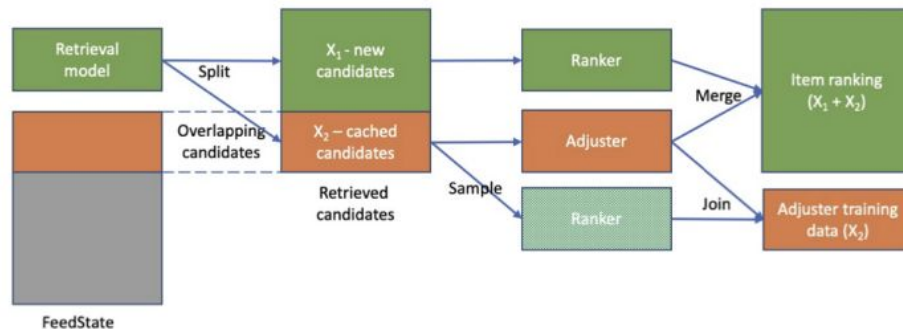


Figure 2: A Delivery Flow with Smart Cache

Recommendation system lifecycle

girafe
ai

12



Features logging

- It is necessary to train the model on the data recorded during the operation of the real system
- Collecting data on historical casts is guaranteed to lead to a discrepancy in values when training and applying models
- Errors in calculations will give rise to incorrect signs with leaks of the target variable ("face")



Reproducible learning

Always versioning your experiments

- Data
 - Parameters of models and training
 - Executable code
-
- And of course fix Random Seed (I advise 42)

Username: **vkokhtev**

Date: 2021-10-21 10:22:35

Status: Completed

Tags: **general** **general_ranking**

Model Name: CatBoost-YetiRank

Model Options:  {

"border-count" : 128

"depth" : 8

"iterations" : 1000

"l2-leaf-reg" : 0.0167177222323797

"learning-rate" : 0.2599701978545636

}

Dataset Name: **CENSORED**

Experiment: 735fdf4b-bf92-44b1-b79b-e2d6cfbd9a86



Metrics monitoring

- Metrics from the last lecture must be constantly monitored
- Model quality metrics

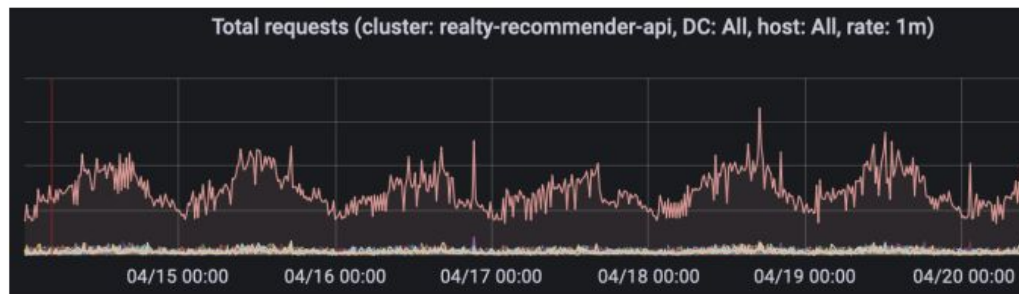
Technical

- Number of requests, response time
- Values of features and responses of models

Количество пользователей, смотревших сниппеты



Total requests (cluster: realty-recommender-api, DC: All, host: All, rate: 1m)



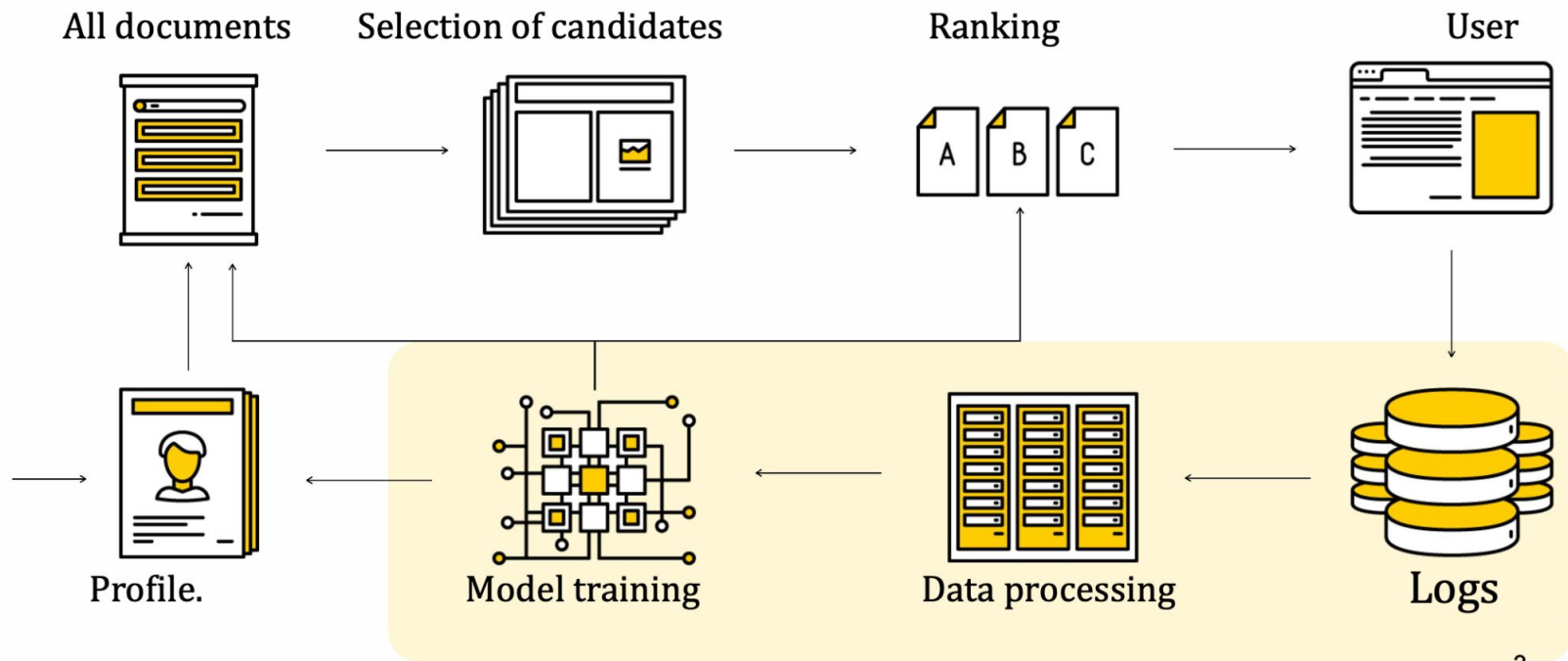


Regular models retraining

- Even if you have received a good recommendation model once, it will become outdated over time. Therefore, it is necessary to periodically train/retrain models on fresh data
- You can trust automation and run each regular model in an A/B experiment, and if the main metrics are stable or improved, include them for all users



Resume



Thanks for attention!

Questions?

girafe
ai

